

点云

高林

中国科学院计算技术研究所

二维图像获取



图像拍摄设备

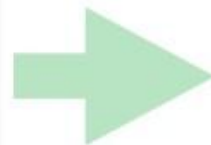
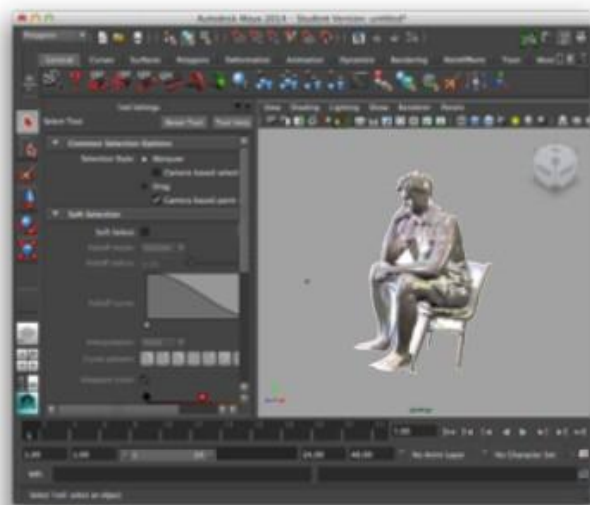
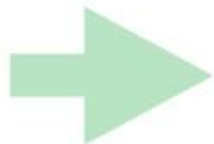


图像处理与编辑



图像打印

三维数据获取



三维扫描设备
Kinect和激光扫描仪等

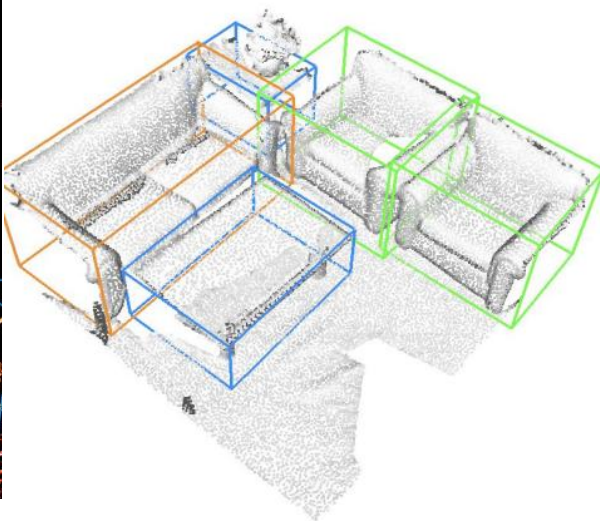
三维处理与编辑

三维打印机

应用范围



无人驾驶



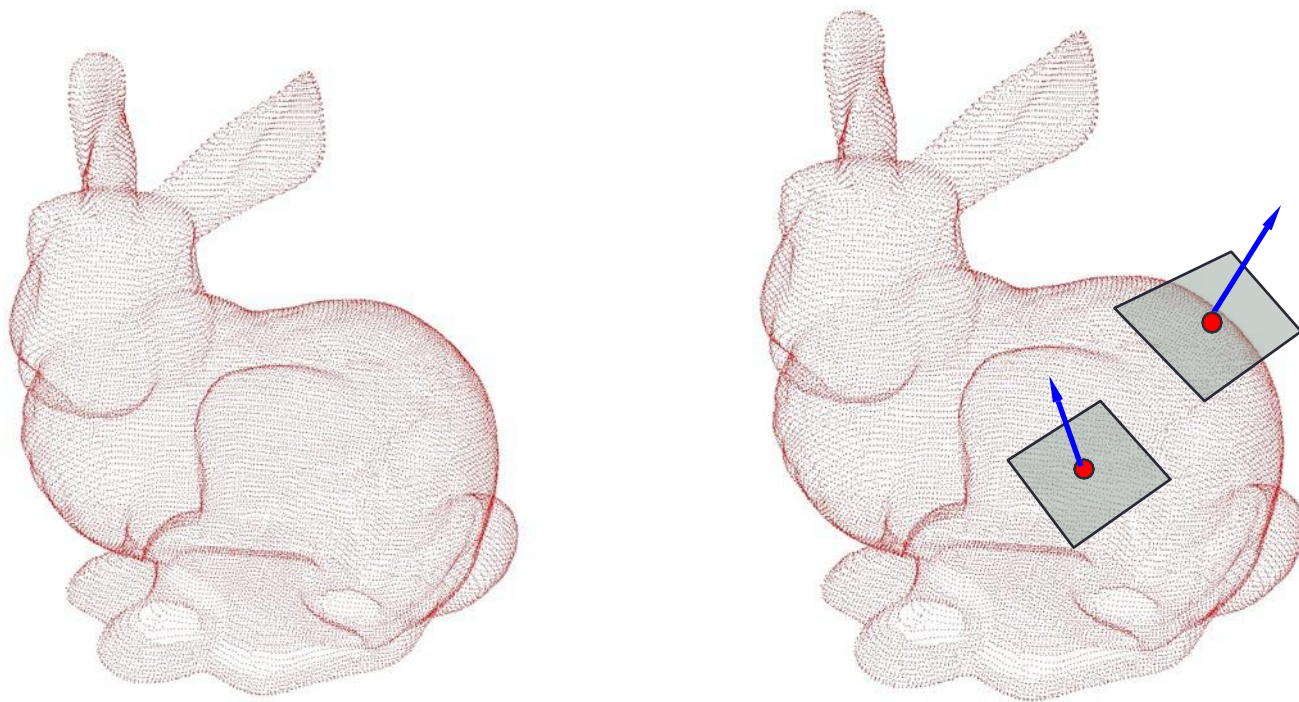
场景感知



三维重建

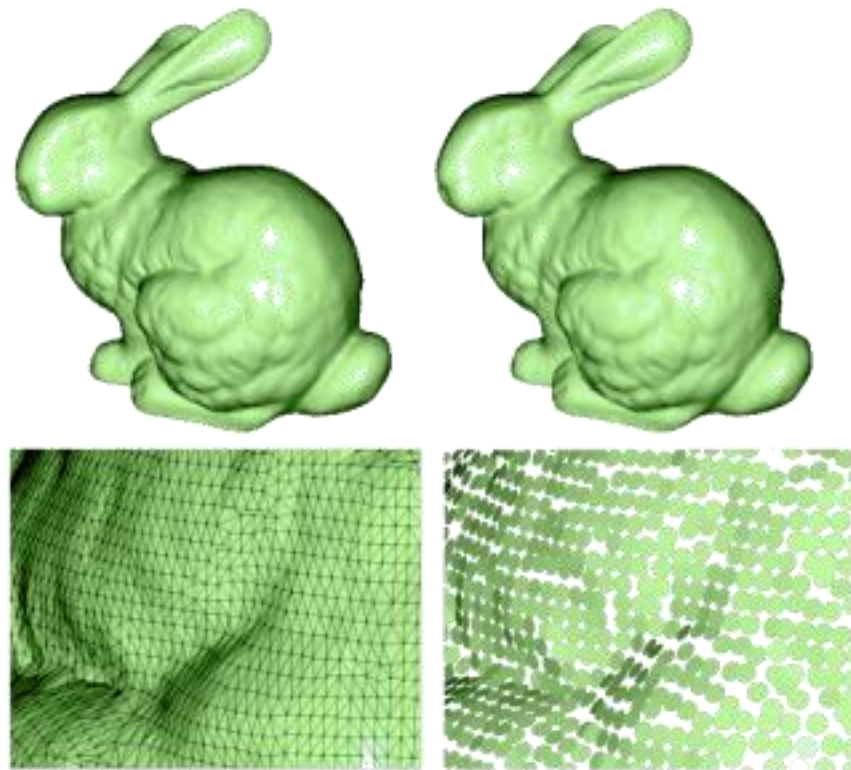
点云 Point Clouds

- 最简单的表示: 只有点, 没有连接性。
- (x, y, z) 坐标的集合, 可能有法向信息。



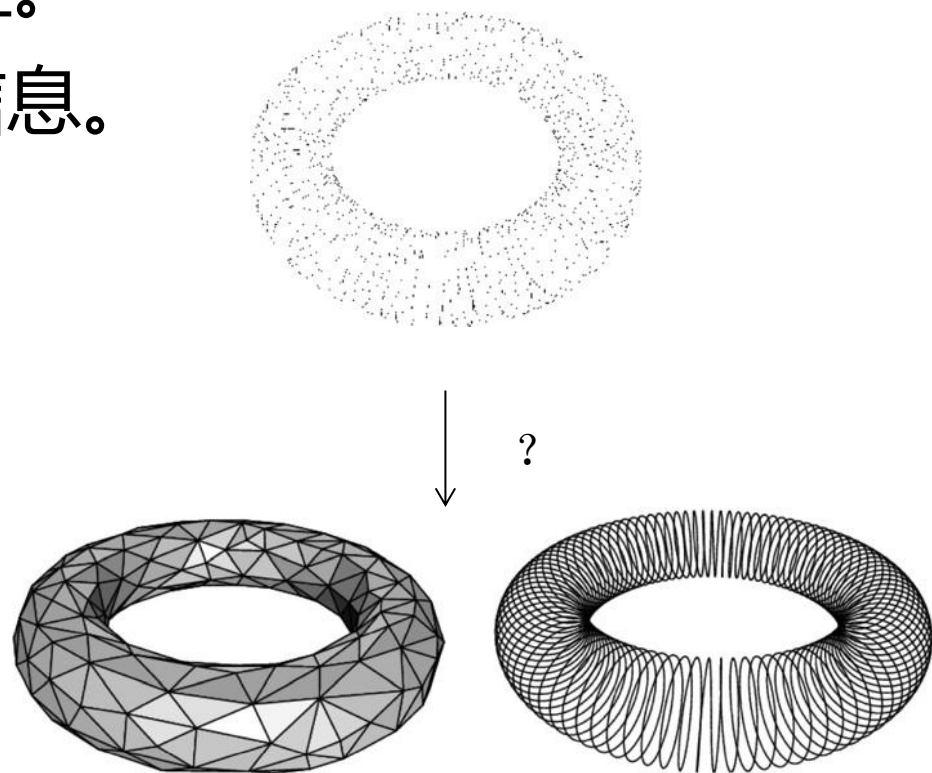
点云 Point Clouds

- 最简单的表示: 只有点, 没有连接性。
- (x, y, z) 坐标的集合, 可能有法向信息。
- 有方向的点成为 surfels (面元)



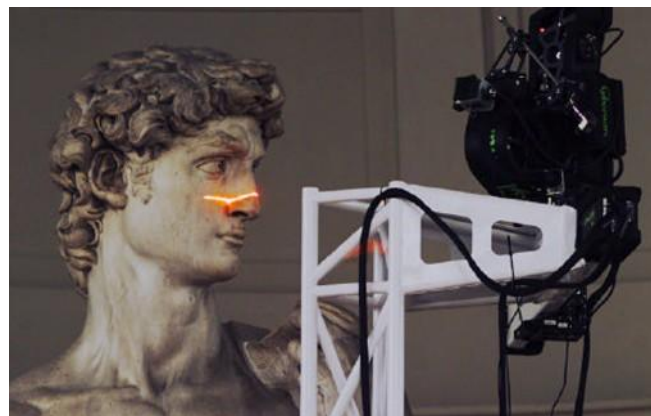
点云 Point Clouds

- 最简单的表示: 只有点, 没有连接性。
- (x, y, z) 坐标的集合, 可能有法向信息。
- 有方向的点成为 surfels (面元)
- 局限:
 - ◆ 无法进行简化和细分操作
 - ◆ 无法直接进行平滑渲染
 - ◆ 无拓扑信息



为什么选择点云？

- 通常情况下，这是唯一可用的三维数据格式
- 几乎所有的3D扫描设备都会产生点云



Time of Flight 激光扫描仪

1. 发出短波激光脉冲
2. 捕捉反射
3. 测量一下回来的时间

$$D = cT/2 \quad c: \text{光速} (\approx 299\,792\,458 \text{ m/s})$$

4. 需要高精度时钟:例如1GHz可以达到0.15m (15cm)的精度。



Time of Flight 激光扫描仪

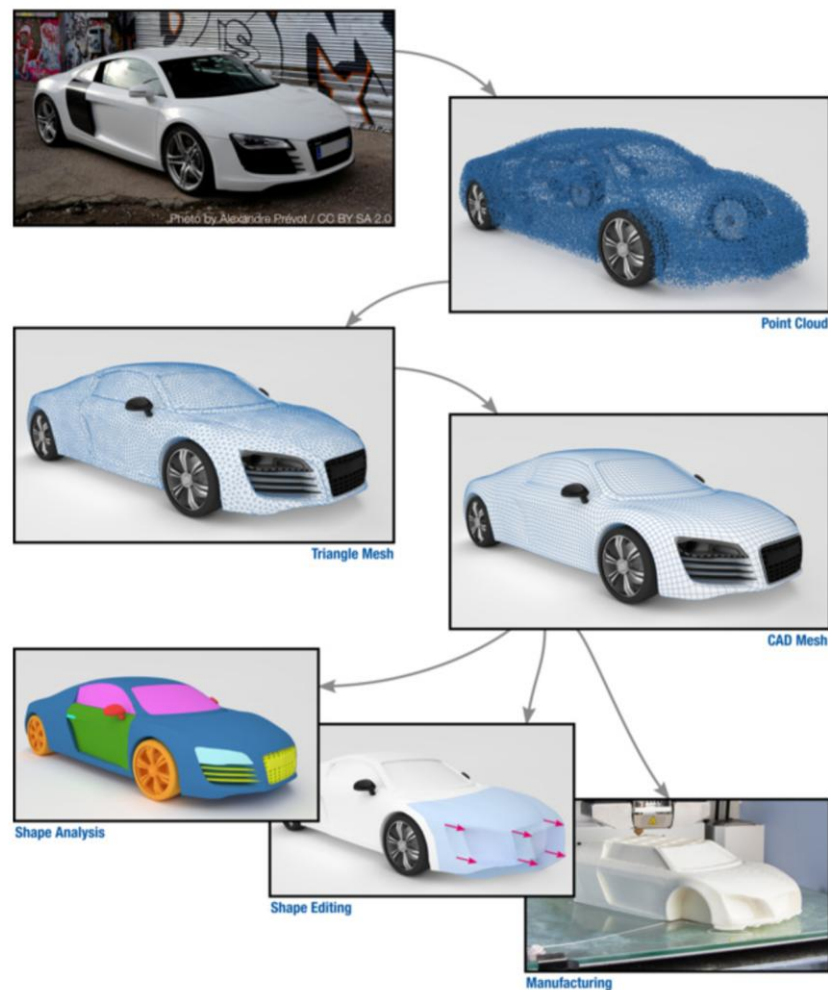
1. 发出短波激光脉冲
2. 捕捉反射
3. 测量一下回来的时间
4. 需要高精度时钟
5. 主要优点：可用于远距离测量
6. 可用于地形扫描



Time of Flight 激光扫描仪



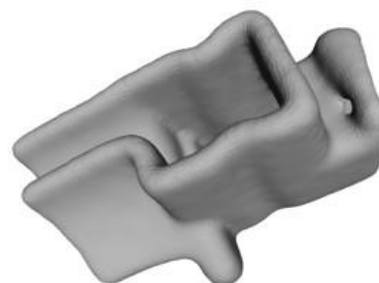
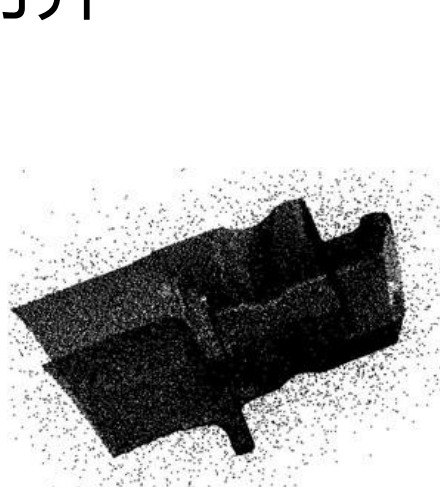
经典的扫描和重建流程



三维点云的处理

■ 通常，形状的点云抽样对于大多数应用程序来说是不够的。加工主要阶段：

1. 形状扫描
2. 如果有多次扫描，将它们对齐
3. 平滑-去除局部噪声
4. 估计表面法线
5. 表面重建



为什么做注册

- 给定(至少)两个具有部分重叠几何形状的图形，找出它们之间的对齐方式。

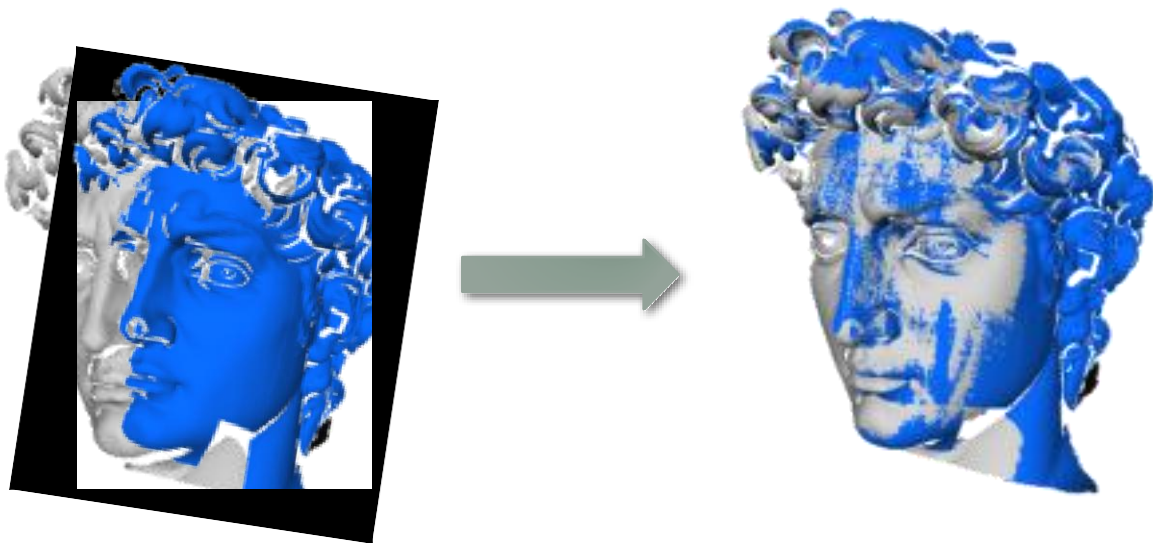


为什么做注册

- 几何分析中的基本问题
- 出现在许多形状分析应用程序中
- ICP: 计算机图形学和计算几何学中最著名的算法之一。广泛应用于工业领域。

局部对齐

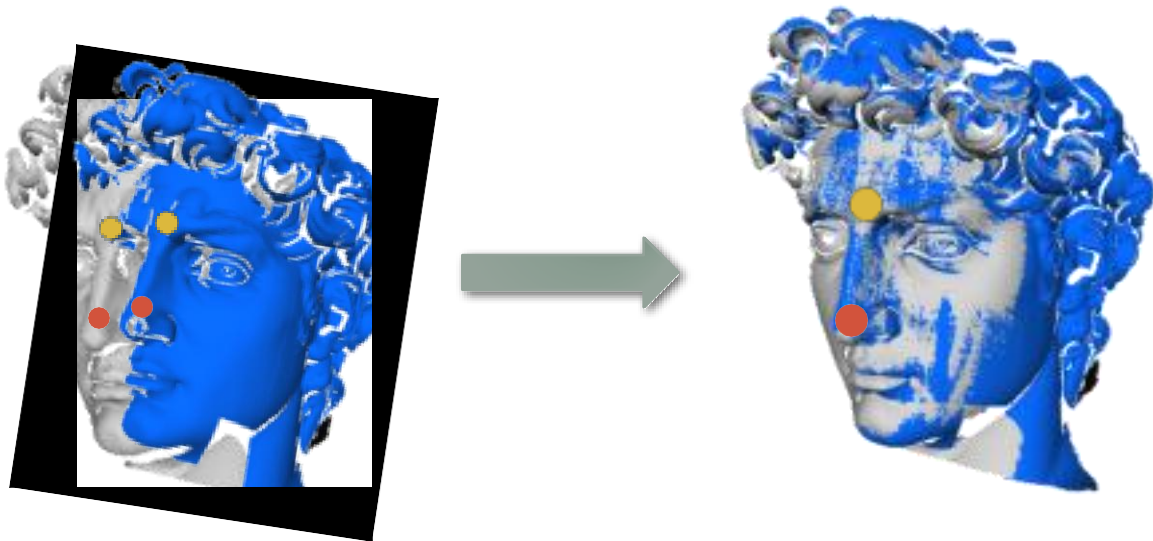
- 注册问题的最简单实例



- 给定两个大致对齐的形状(例如, 一个人), 我们希望找到最优的变形

局部对齐

- 什么样的结果算是一个比较好的注册效果？



- 直觉： 变换后对应点相近。
- 问题： 我们不知道对应的点是什么。
我们不知道最佳的排列方式。

迭代最近点算法 (ICP)

- 方法: 查找对应和查找转换之间的迭代:

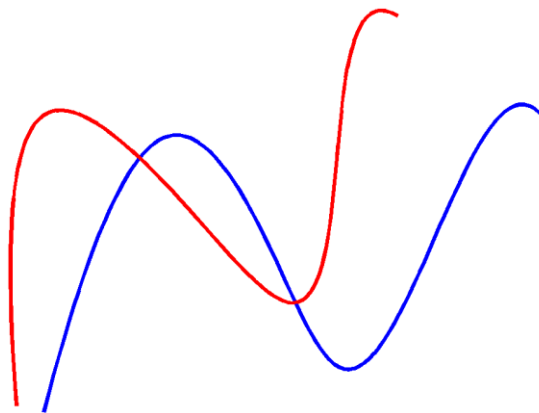


- 给定一对形状, X 和 Y , 迭代:
 1. 对任意 $x_i \in X$, 找到最近点 $y_i \in Y$.
 2. 求解变形参数旋转 R 和平移 t , 最小化

$$\sum_{i=1}^N \|Rx_i + t - y_i\|_2^2$$

迭代最近点算法

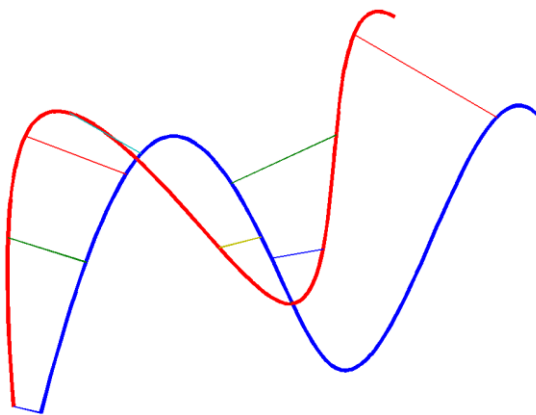
- 方法: 查找对应和查找转换之间的迭代:



- 给定一对形状, X 和 Y , 迭代:
- 对任意 $x_i \in X$, 找到最近点 $y_i \in Y$.
- 求解变形参数旋转 R 和平移 t , 最小化 $\sum_{i=1}^N \|Rx_i + t - y_i\|_2^2$

迭代最近点算法

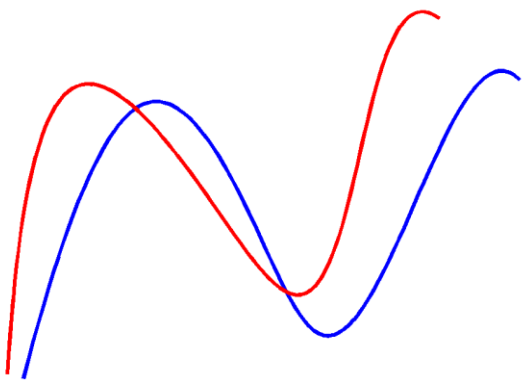
- 方法: 查找对应和查找转换之间的迭代:



- 给定一对形状, X 和 Y , 迭代:
- 对任意 $x_i \in X$, 找到最近点 $y_i \in Y$.
- 求解变形参数旋转 R 和平移 t , 最小化 $\sum_{i=1}^N \|Rx_i + t - y_i\|_2^2$

迭代最近点算法

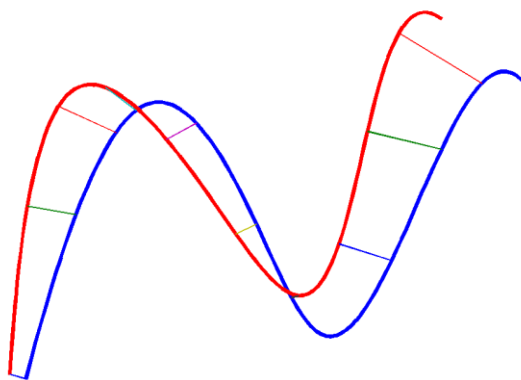
- 方法: 查找对应和查找转换之间的迭代:



- 给定一对形状, X 和 Y , 迭代:
- 对任意 $x_i \in X$, 找到最近点 $y_i \in Y$.
- 求解变形参数旋转 R 和平移 t , 最小化 $\sum_{i=1}^N \|Rx_i + t - y_i\|_2^2$

迭代最近点算法

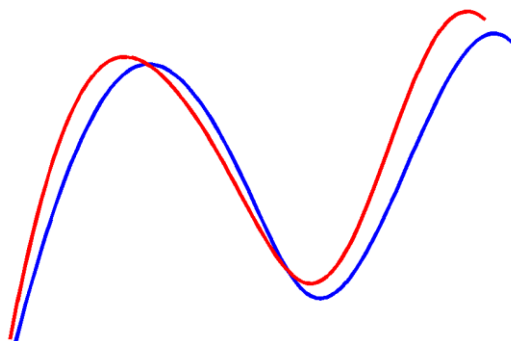
- 方法: 查找对应和查找转换之间的迭代:



- 给定一对形状, X 和 Y , 迭代:
- 对任意 $x_i \in X$, 找到最近点 $y_i \in Y$.
- 求解变形参数旋转 R 和平移 t , 最小化 $\sum_{i=1}^N \|Rx_i + t - y_i\|_2^2$

迭代最近点算法

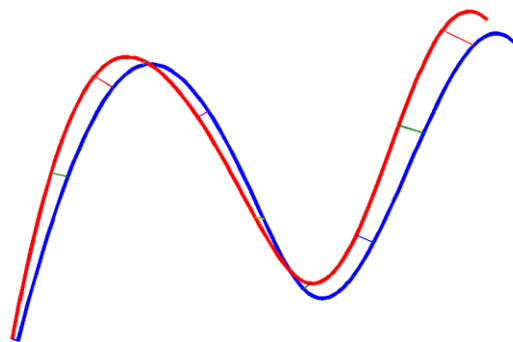
- 方法: 查找对应和查找转换之间的迭代:



- 给定一对形状, X 和 Y , 迭代:
- 对任意 $x_i \in X$, 找到最近点 $y_i \in Y$.
- 求解变形参数旋转 R 和平移 t , 最小化 $\sum_{i=1}^N \|Rx_i + t - y_i\|_2^2$

迭代最近点算法

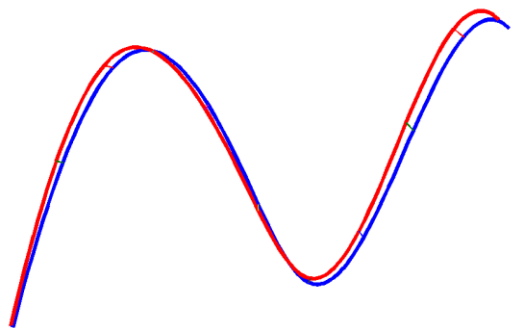
- 方法: 查找对应和查找转换之间的迭代:



- 给定一对形状, X 和 Y , 迭代:
- 对任意 $x_i \in X$, 找到最近点 $y_i \in Y$.
- 求解变形参数旋转 R 和平移 t , 最小化 $\sum_{i=1}^N \|Rx_i + t - y_i\|_2^2$

迭代最近点算法

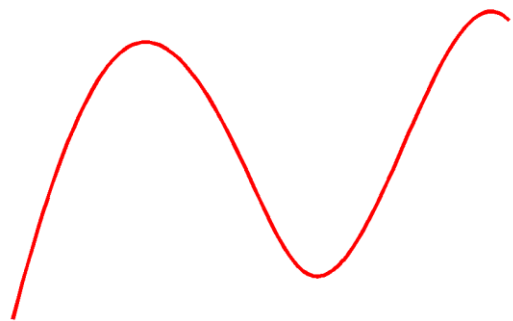
- 方法: 查找对应和查找转换之间的迭代:



- 给定一对形状, X 和 Y , 迭代:
- 对任意 $x_i \in X$, 找到最近点 $y_i \in Y$.
- 求解变形参数旋转 R 和平移 t , 最小化 $\sum_{i=1}^N \|Rx_i + t - y_i\|_2^2$

迭代最近点算法

- 方法: 查找对应和查找转换之间的迭代:

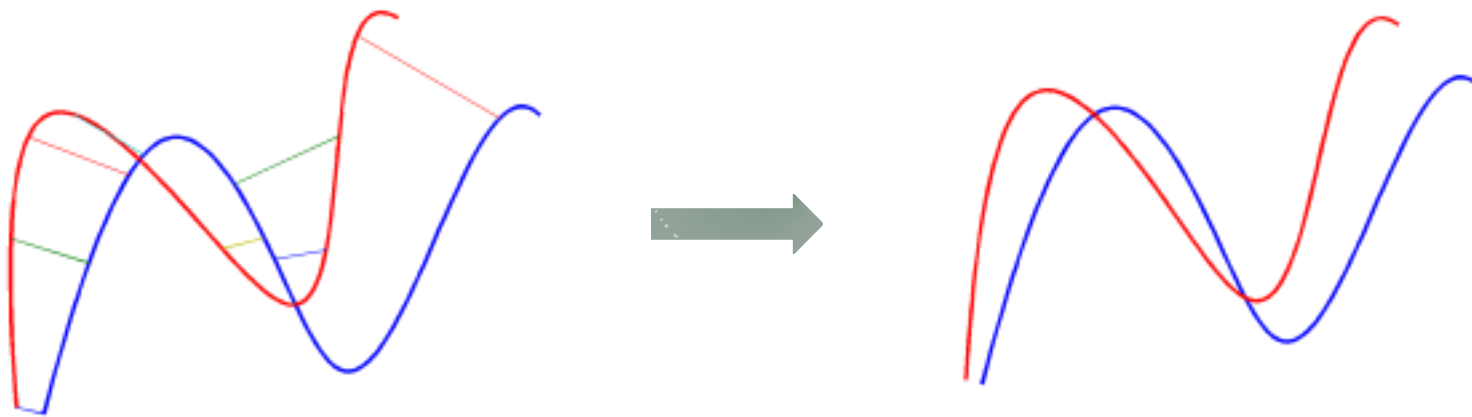


- 给定一对形状, X 和 Y , 迭代:
- 对任意 $x_i \in X$, 找到最近点 $y_i \in Y$.
- 求解变形参数旋转 R 和平移 t , 最小化 $\sum_{i=1}^N \|Rx_i + t - y_i\|_2^2$

迭代最近点算法

■ 两个主要的计算：

1. 最近点计算
2. 最优变换计算



ICP: 计算最近点

- 最近点:

$$y_i = \arg \min_{y \in Y} \|y - x_i\|$$

- 如何有效的找到最近点?

- 计算复杂度 $O(MN)$

M 表示 X 上的顶点数目, N 表示 Y 上的顶点数量

ICP: 最优变换

■ 问题公式化:

1. 给定两个点集 $\{x_i\}, \{y_i\}, i=1..n$ in $\mathbb{R}^3$. 最小化能连函数求解刚性变换参数 R, t

$$\sum_{i=1}^N \|Rx_i + t - y_i\|_2^2$$

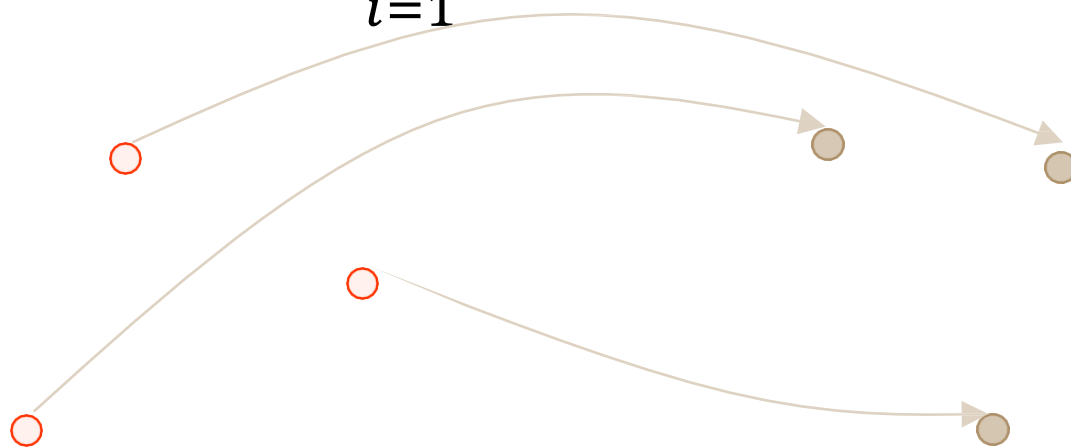


ICP: 最优变换

■ 问题公式化:

1. 给定两个点集 $\{x_i\}, \{y_i\}, i=1..n$ in $\mathbb{R}^3$. 最小化能连函数求解刚性变换参数 R, t

$$\sum_{i=1}^N \|Rx_i + t - y_i\|_2^2$$



ICP: 最优变换

■ 问题公式化:

2. 带旋转矩阵的封闭解

1. 重建: $C = \sum_{i=1}^N (y_i - \mu^Y)(x_i - \mu^X)^T$, 同时同时 $\mu^X = \frac{1}{N} \sum_i x_i$, $\mu^Y = \frac{1}{N} \sum_i y_i$
2. 对 C 进行SVD分解: $C = U\Sigma V^T$
 1. 如果 $\det(UV^T) = 1$, $R_{opt} = UV^T$
 2. 否则 $R_{opt} = U\tilde{\Sigma}V^T$, $\tilde{\Sigma} = \text{diag}(1, 1, \dots, -1)$
3. $t_{opt} = \mu^Y - R_{opt}\mu^X$

迭代最近点算法

■ 给定一对形状, X 和 Y , 迭代:

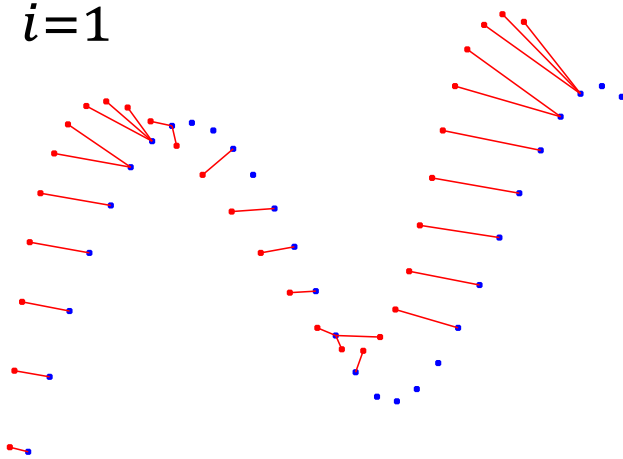
1. 对任意 $x_i \in X$, 找到最近点 $y_i \in Y$.
2. 求解变形参数旋转 R 和平移 t , 最小化 $\sum_{i=1}^N \|Rx_i + t - y_i\|_2^2$

■ 收敛:

1. 在每次迭代中 $\sum_{i=1}^N d^2(x_i, Y)$ 递减
2. 收敛到局部最小化
3. 比较好的初始化: 全局最小化

迭代最近点算法

- 给定一对形状， X 和 Y ，迭代：
 1. 对任意 $x_i \in X$ ，找到最近点 $y_i \in Y$.
 2. 求解变形参数旋转 R 和平移 t ，最小化

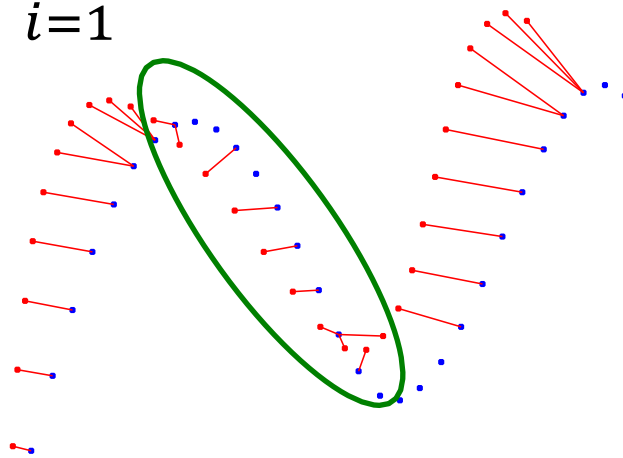
$$\sum_{i=1}^N \|Rx_i + t - y_i\|_2^2$$


迭代最近点算法

- 给定一对形状, X 和 Y , 迭代:
 1. 对任意 $x_i \in X$, 找到最近点 $y_i \in Y$.
 2. 求解变形参数旋转 R 和平移 t , 最小化

$$\sum_{i=1}^N \|Rx_i + t - y_i\|_2^2$$

问题:
非均匀采样



迭代最近点算法

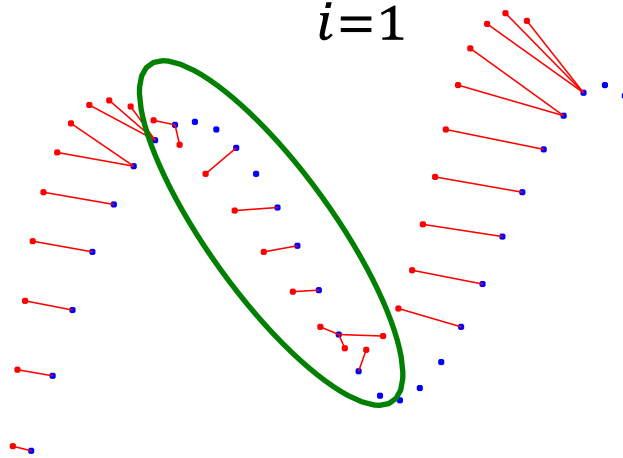
■ 给定一对形状， X 和 Y ，迭代：

1. 对任意 $x_i \in X$ ，找到最近点 $y_i \in Y$ 。
2. 求解变形参数旋转 R 和平移 t ，最小化

$$\sum_{i=1}^N d(Rx_i + t, P(y_i))^2 = \sum_{i=1}^N \left((Rx_i + t - y_i)^T n_{y_i} \right)^2$$

求解：

最小化点到切
平面的距离



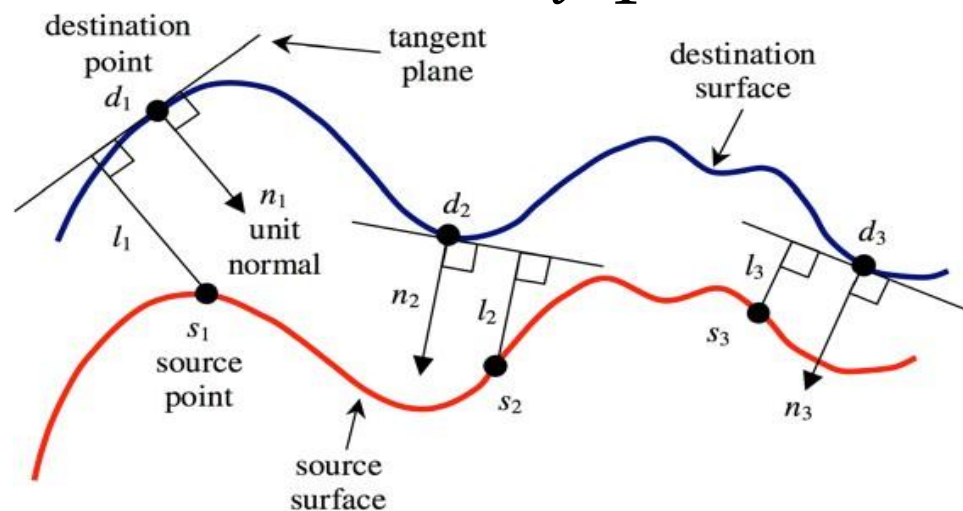
迭代最近点算法

■ 给定一对形状， X 和 Y ，迭代：

1. 对任意 $x_i \in X$ ，找到最近点 $y_i \in Y$ 。
2. 求解变形参数旋转 R 和平移 t ，最小化

$$\sum_{i=1}^N d(Rx_i + t, P(y_i))^2 = \sum_{i=1}^N \left((Rx_i + t - y_i)^T n_{y_i} \right)^2$$

求解：
最小化点到切
平面的距离



迭代最近点算法

■ 给定一对形状, X 和 Y , 迭代:

1. 对任意 $x_i \in X$, 找到最近点 $y_i \in Y$.
2. 求解变形参数旋转 R 和平移 t , 最小化

$$R_{opt}, t_{opt} = \arg \min_{R^T R = Id, t} \sum_{i=1}^N \left((Rx_i + t - y_i)^T n_{y_i} \right)^2$$

问题: 如何最小化误差

挑战: 虽然误差是二次的(线性导数), 但旋转矩阵的空间不是线性的

难点: 并无闭式解

迭代最近点算法

■ 给定一对形状, X 和 Y , 迭代:

1. 对任意 $x_i \in X$, 找到最近点 $y_i \in Y$.
2. 求解变形参数旋转 R 和平移 t , 最小化

$$R_{opt}, t_{opt} = \arg \min_{R^T R = Id, t} \sum_{i=1}^N \left((Rx_i + t - y_i)^T n_{y_i} \right)^2$$

常见的解决方案:

线性化旋转。假设旋转角度很小

$$Rx_i \approx x_i + r \times x_i \times \alpha$$

r : 维度
 $\|r\|_2$: 旋转角

$$R(r, \alpha)x_i = x_i \cos \alpha + (r \times x_i) \sin \alpha + r(r^T x_i)(1 - \cos \alpha)$$

一阶导近似: $\sin \alpha \approx \alpha, \cos \alpha = 1$

迭代最近点算法

■ 给定一对形状, X 和 Y , 迭代:

1. 对任意 $x_i \in X$, 找到最近点 $y_i \in Y$.
2. 求解变形参数旋转 R 和平移 t , 最小化

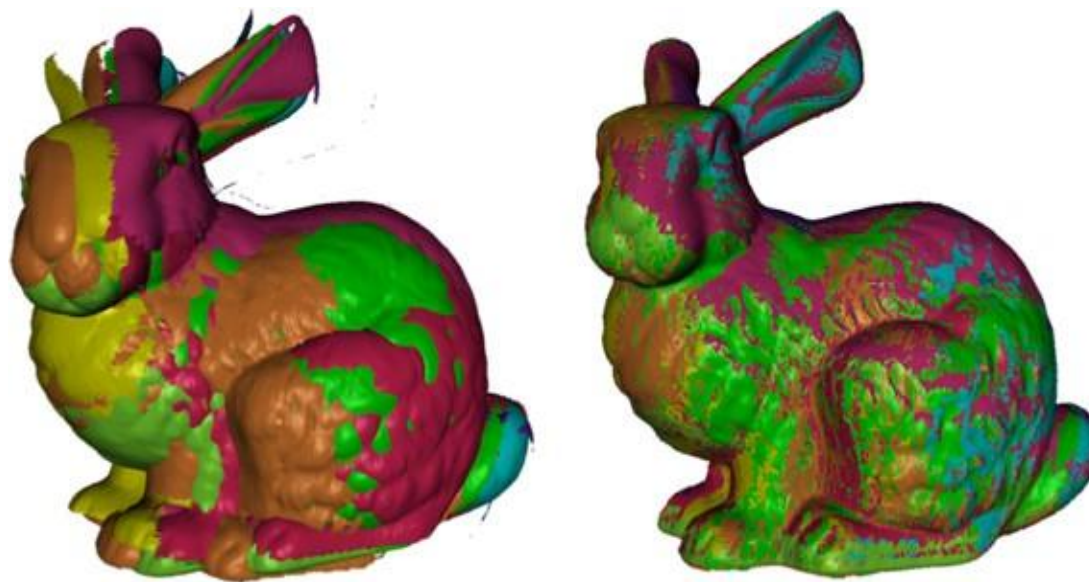
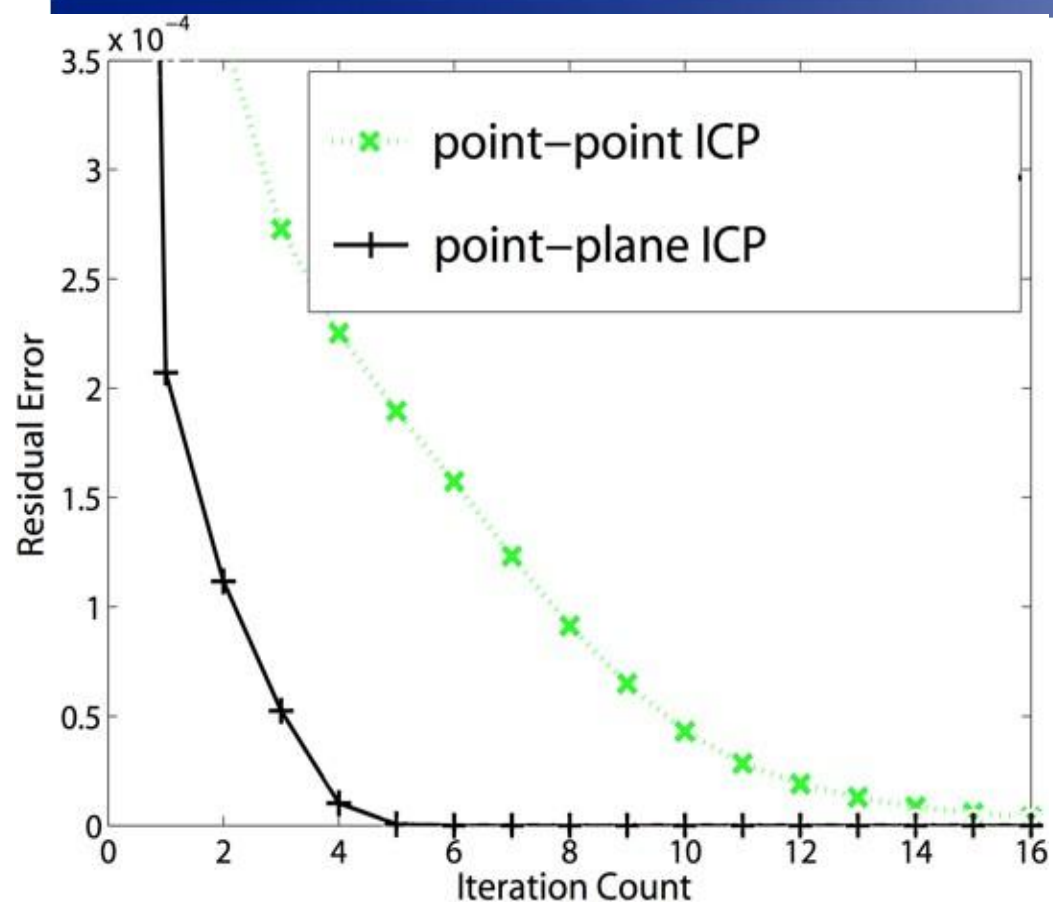
$$E(r, t) = \sum_{i=1}^N \left((x_i + r \times x_i + t - y_i)^T n_{y_i} \right)^2$$

■ 当设 $\frac{\partial}{\partial r} E(r, t) = 0$ 和 $\frac{\partial}{\partial t} E(r, t) = 0$ 时, 将产生一个 6×6 的线性系统

$$Ax = b$$

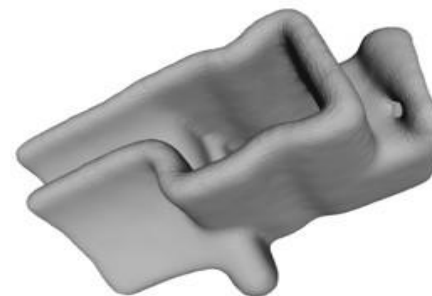
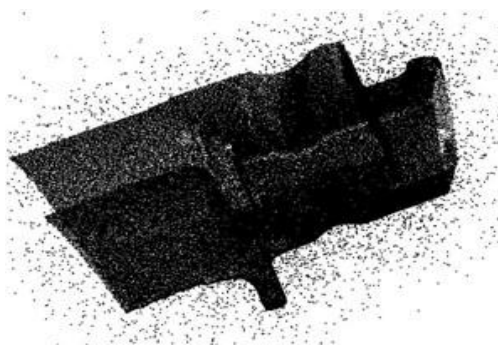
$$x = \begin{pmatrix} r \\ t \end{pmatrix} \quad A = \sum \begin{pmatrix} x_i \times n_{y_i} \\ n_{y_i} \end{pmatrix} \begin{pmatrix} x_i \times n_{y_i} \\ n_{y_i} \end{pmatrix}^T \quad b = \sum (y_i - x_i)^T n_{y_i} n_{y_i}$$

迭代最近点算法



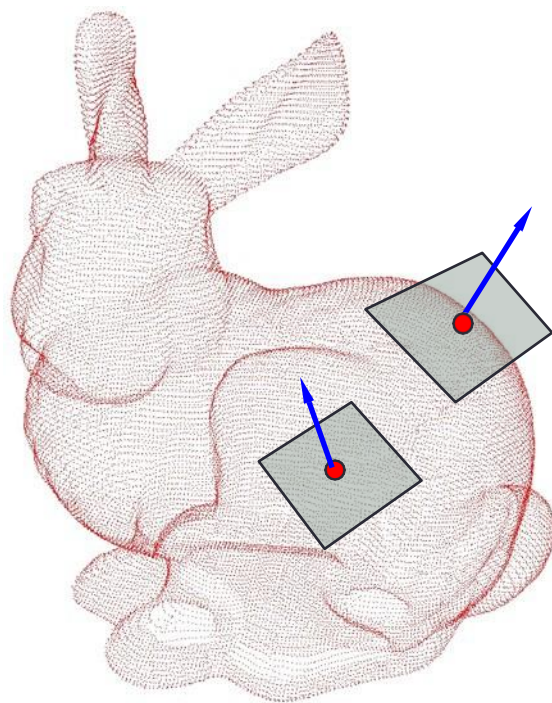
三维点云的处理

- 通常，形状的点云抽样对于大多数应用程序来说是不够的。加工主要阶段：
 1. 形状扫描
 2. 如果有多次扫描，将它们对齐
 3. 平滑-去除局部噪声
 4. 估计表面法线
 5. 表面重建



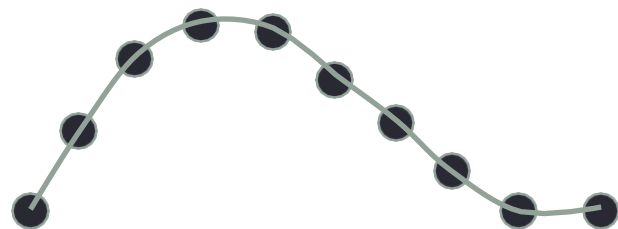
法向估计和去噪

■ 点云处理中的基本问题



法向估计

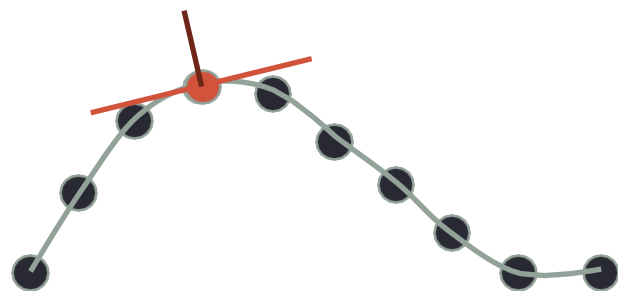
假设我们有一个干净的表面样本。



目标：找到切线方向的最佳近似值，从而找到直线的法线

法向估计

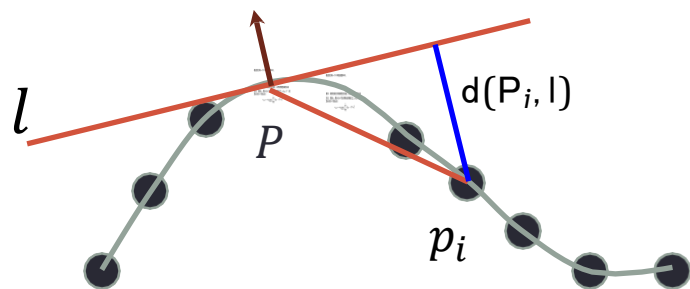
假设我们有一个干净的表面样本。



目标：找到切线方向的最佳近似值，从而找到直线的法线

法向估计

假设我们有一个干净的表面样本。



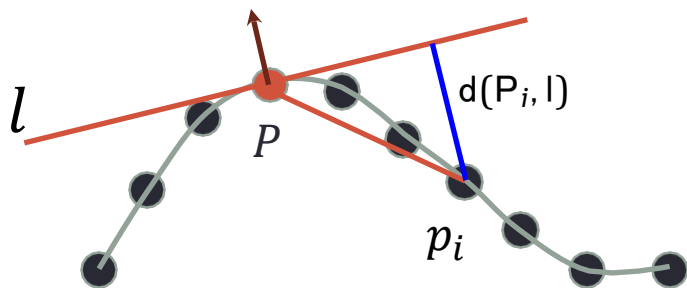
目标：找到切线方向的最佳近似值，从而找到直线的法线

方法：找到 n ，最小化 k 个顶点集合的能量 $\sum_{i=1}^k d(p_i, l)^2$ （如顶点 P 的 k 个邻近点）

$$n_{opt} = \arg \min_{\|n\|=1} \sum_{i=1}^k ((p_i - P)^T n)^2$$

法向估计

假设我们有一个干净的表面样本。

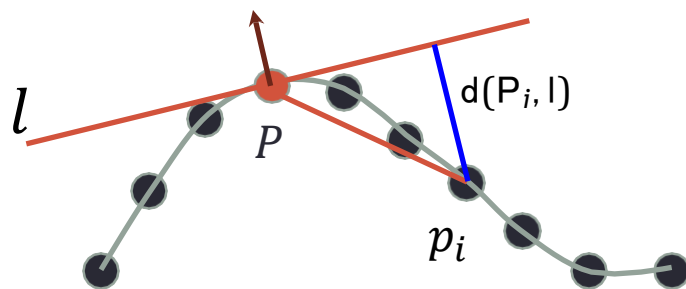


使用拉格朗日乘子：

$$\frac{\partial}{\partial n} \left(\sum_{i=1}^k ((p_i - P)^T n)^2 \right) - \lambda \frac{\partial}{\partial n} (n^T n) = 0$$
$$\sum_{i=1}^k 2(p_i - P)(p_i - P)^T n = 2\lambda n$$

法向估计

假设我们有一个干净的表面样本。



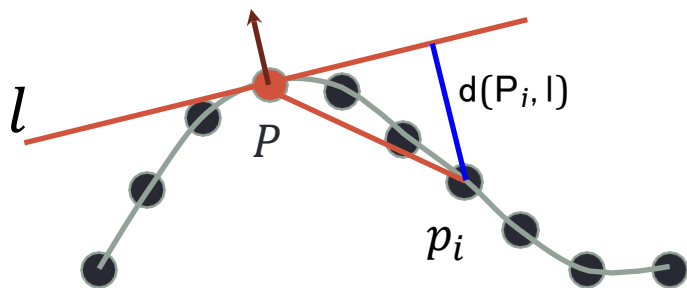
使用拉格朗日乘子：

$$\frac{\partial}{\partial n} \left(\sum_{i=1}^k ((p_i - P)^T n)^2 \right) - \lambda \frac{\partial}{\partial n} (n^T n) = 0$$

$$\left(\sum_{i=1}^k (p_i - P)(p_i - P)^T \right) n = \lambda n \quad \rightarrow \quad Cn = \lambda n$$

法向估计

假设我们有一个干净的表面样本。



法向量 n 必须是矩阵的一个特征向量：

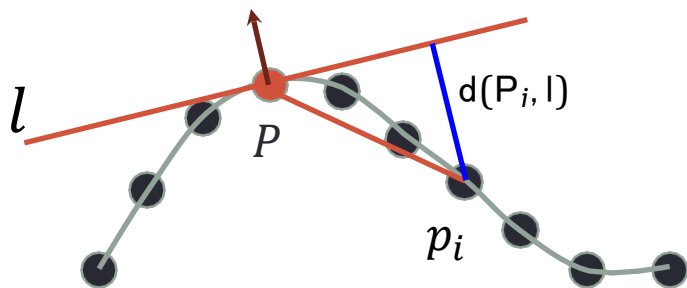
$$Cn = \lambda n \quad C = \sum_{i=1}^k (p_i - P)(p_i - P)^T$$

因此，

$$n_{opt} = \arg \min_{\|n\|=1} \sum_{i=1}^k ((p_i - P)^T n)^2 = \arg \min_{\|n\|=1} n^T C n$$

法向估计

假设我们有一个干净的表面样本。



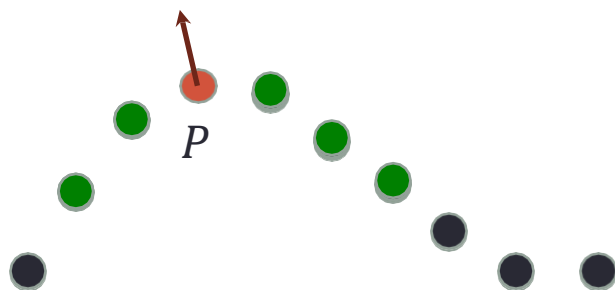
法向量 n 必须是矩阵的一个特征向量：

$$Cn = \lambda n \quad C = \sum_{i=1}^k (p_i - P)(p_i - P)^T$$

而且, n_{opt} 必须是对应于 C 的最小特征值的特征向量。

法向估计

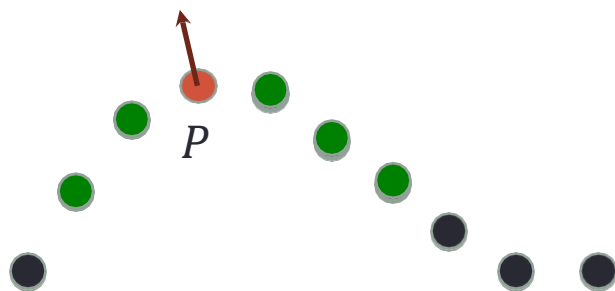
■ 方法 (PCA) :



1. 给定点云上的点 P , 找到最邻近的 k 个点
2. 计算 $C = \sum_{i=1}^k (p_i - P)(p_i - P)^T$
3. n : 对应于 C 的最小特征值的特征向量

法向估计

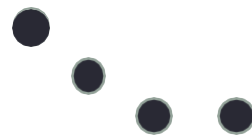
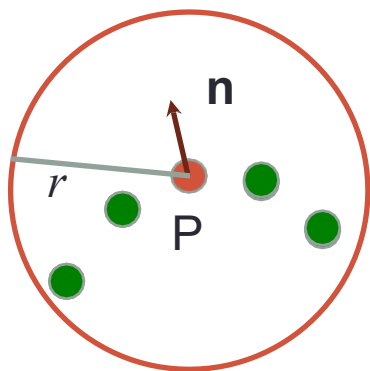
■ 方法 (PCA) :



1. 给定点云上的点 P , 找到最邻近的 k 个点
2. 计算 $C = \sum_{i=1}^k (p_i - P)(p_i - P)^T$
3. n : 对应于 C 的最小特征值的特征向量

变换: 使用 $C = \sum_{i=1}^k (p_i - \bar{P})(p_i - \bar{P})^T$, $\bar{P} = \frac{1}{k} \sum_{i=1}^k p_i$

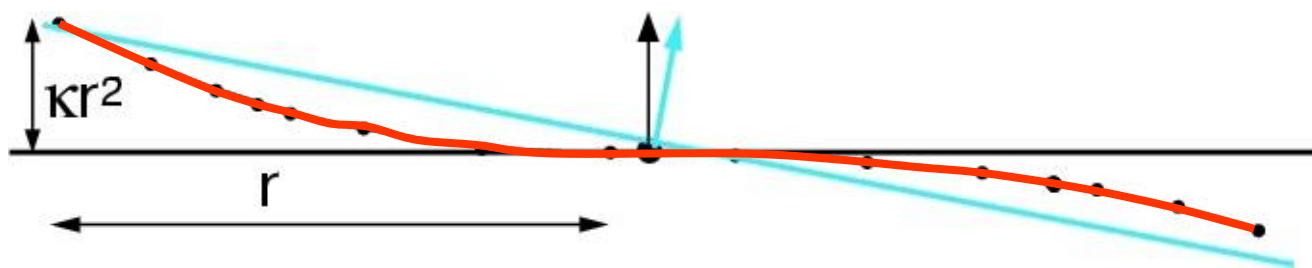
法向估计



- 关键参数: k 由于采样不均匀, 一般固定一个半径为 r 的点, 并使用半径为 r 的球内的所有点。
- 如何选择一个最优的 r ?

法向估计

Curvature effect



- 由于曲率，较大的 r 会导致估计偏差。
- 由于噪声，小 r 会导致误差

法向估计-邻域大小



1x noise

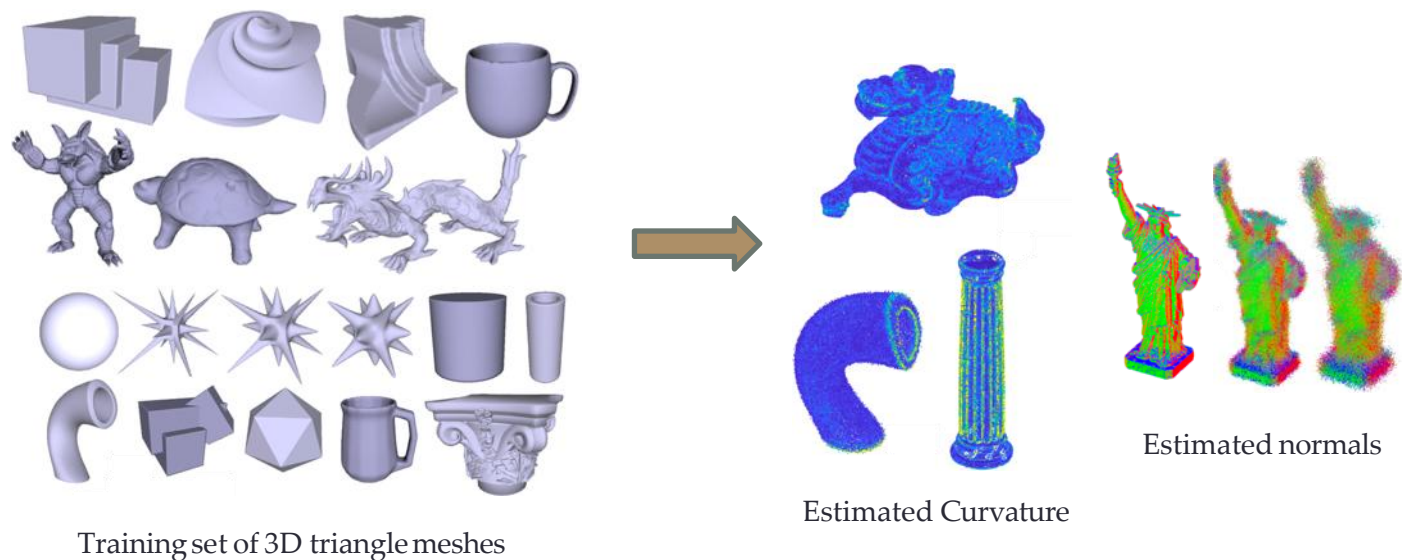


2x noise

source: Mitra
et al. '04

法向和曲率估计

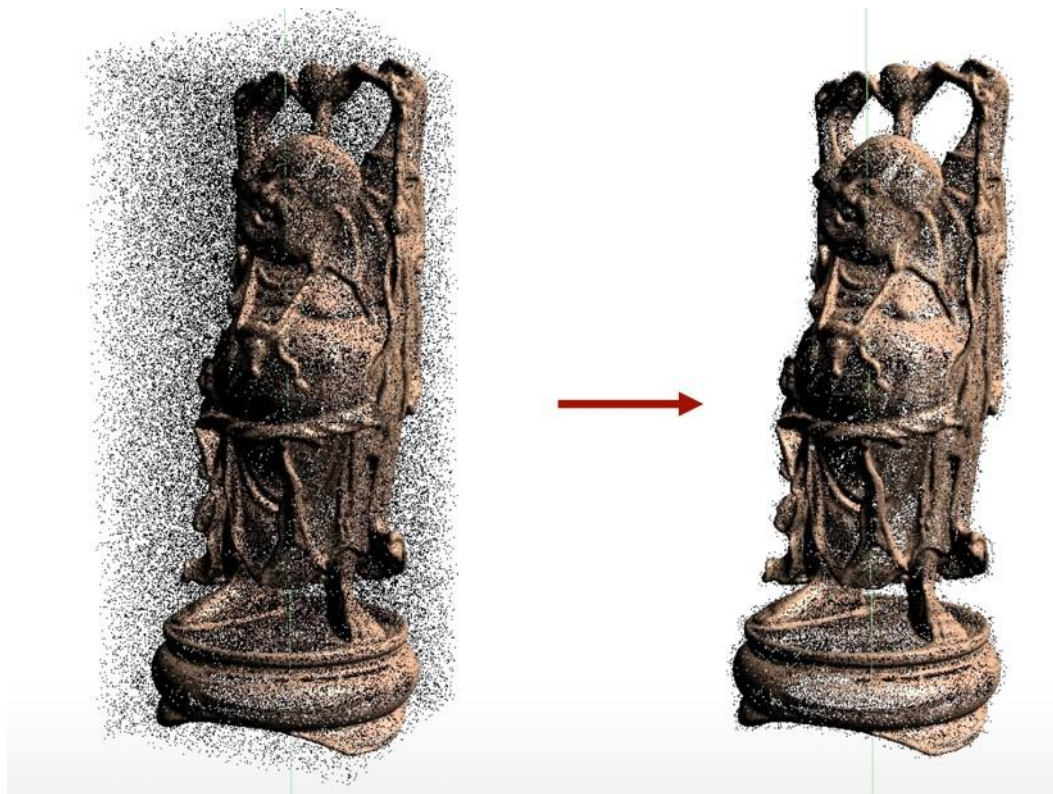
- 使用各种无噪点的三维模型来训练一种基于深度学习的方法来估计法线和曲率



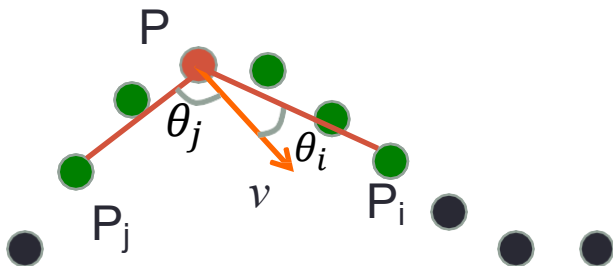
PCPNET: Learning Local Shape Properties from Raw Point Clouds, Proc. Eurographics 2018, Guerrero, Kleiman, O., Mitra

去噪

- 目标: 移除不靠近表面的点。



噪音估计-PCA

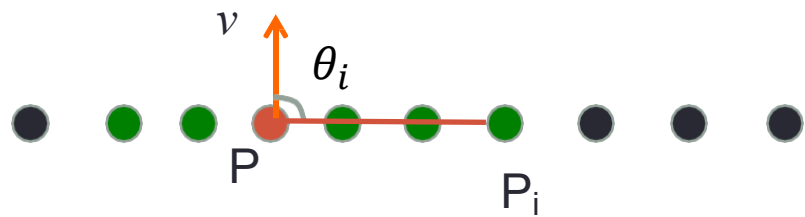


- 由于协方差矩阵: $C = \sum_{i=1}^k (p_i - P)(p_i - P)^T$
- 对于任意向量 v :

$$\begin{aligned} v^T C v &= \sum_{i=1}^k ((p_i - P)^T v)^2 \text{ if } \|v\| = 1 \\ &= \sum_{i=1}^k (\|p_i - P\| \cos \theta_i)^2 \end{aligned}$$

- 直观地说, 最大化每个向量的角度 $(p_i - P)$

噪音估计-PCA

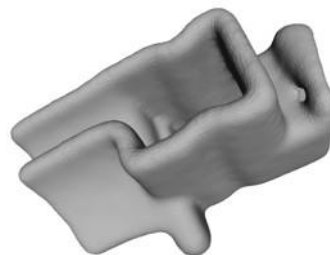
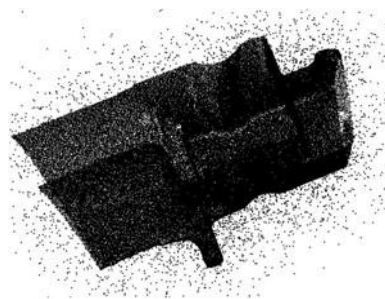


- 如果所有的点都在一条线上, 则 $\min_v v^T C v = \lambda_{min}(C) = 0$, $\lambda_{max}(C)$ 很大
- 存在一个点云无可变性的方向。
- 如果点随机分布, 则: $\lambda_{max}(C) \approx \lambda_{min}(C)$
- 因此, 可以移除超过一定阈值的点 $\frac{\lambda_1}{\lambda_2} > \epsilon$
- 在三维中我们期望两个零特征值, 因此使用多个阈值

三维点云的处理

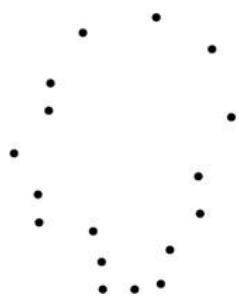
■ 通常，形状的点云抽样对于大多数应用程序来说是不够的。加工主要阶段：

1. 形状扫描
2. 如果有多次扫描，将它们对齐
3. 平滑-去除局部噪声
4. 估计表面法线
5. **表面重建**



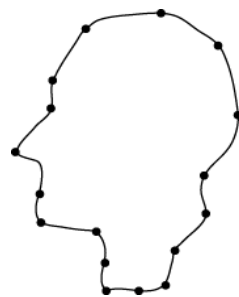
三维点云的重建

- 主要目标:
- 构造点云的多边形(如三角形网格)表示。



点云数据

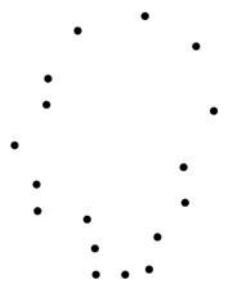
重建算法



曲线 / 曲面

三维点云的重建

- 主要问题：
- 非结构化数据。例如，在2D中，点数是无序的。



点云数据

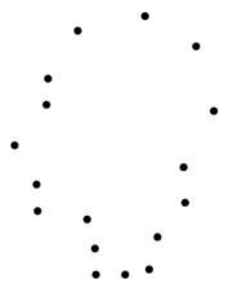
重建算法



曲线 / 曲面

三维点云的重建

- 主要问题：
- 非结构化数据。例如，在2D中，点数是无序的。
- 本质上是一个病态的问题



点云数据

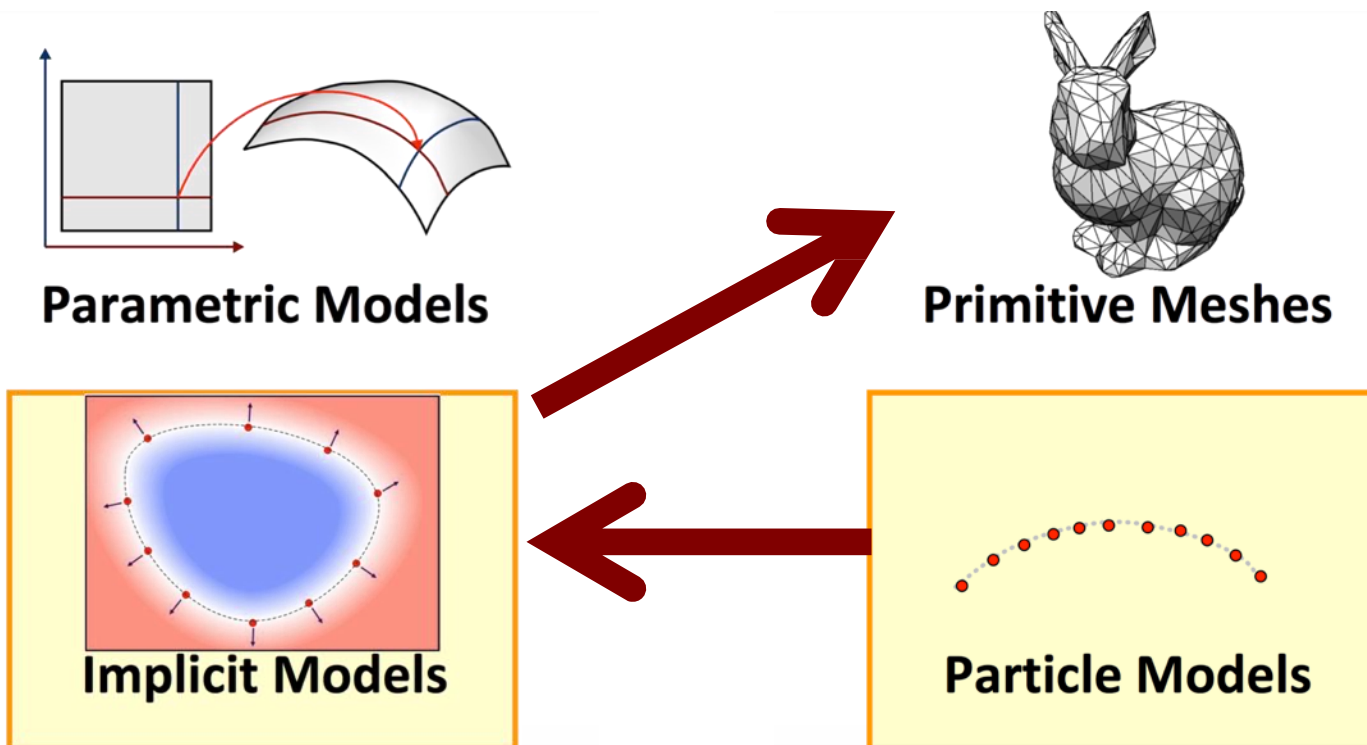
重建算法



曲线 / 曲面

三维点云的重建

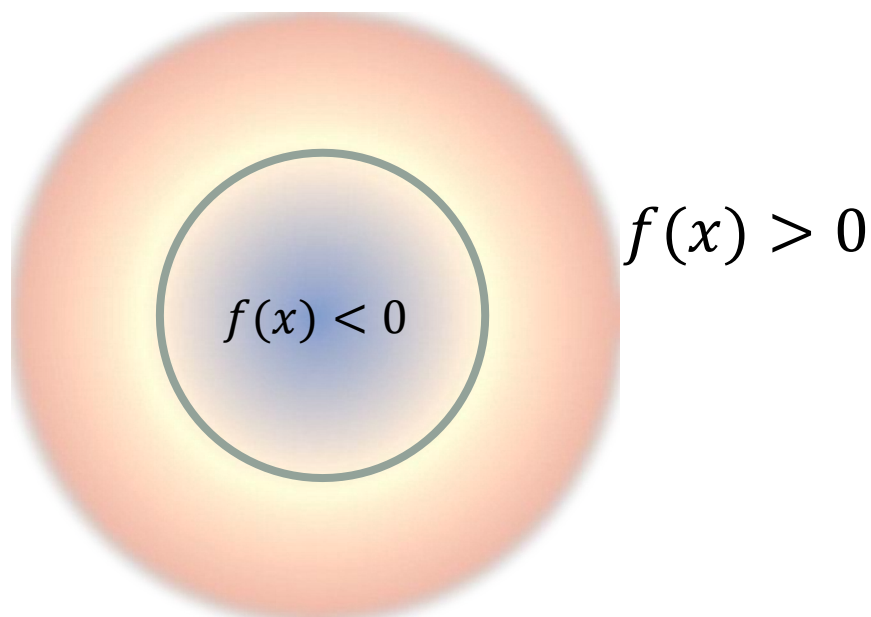
- 通过隐式模型进行重构。



隐式曲面

- 给定函数 $f(x)$, 曲面定义为:

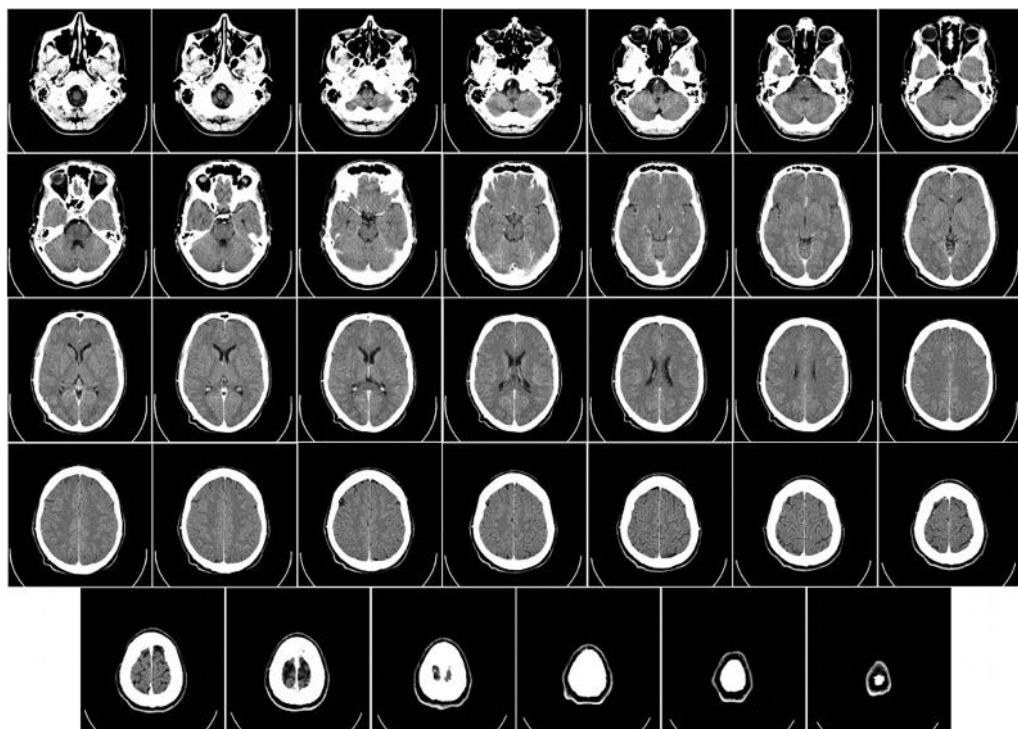
$$\{x, s. t. f(x) = 0\}$$



$$f(x, y) = x^2 + y^2 - r^2$$

隐式曲面

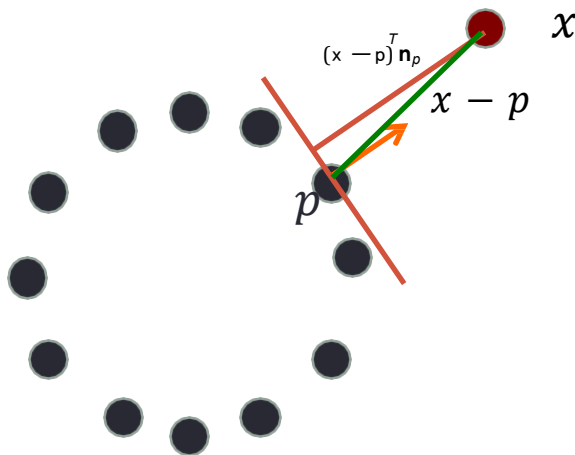
- 一些3D扫描技术(如CT、MRI)自然产生隐式表达



人脑的CT扫描结果

隐式曲面

■ 从点云到隐式表面的转换:

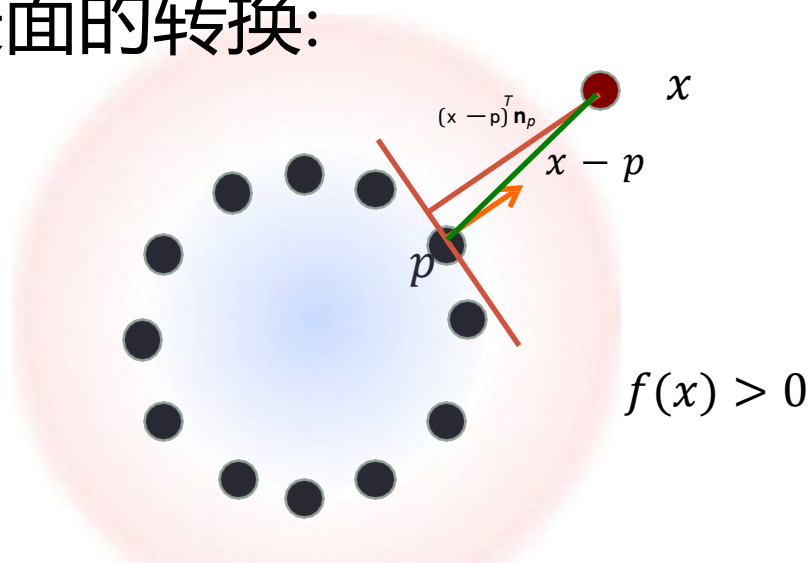


■ 最简单的方法:

1. 给定空间中的点 x , 找出点云数据中最近的点 p 。
2. 集合 $f(x) = (x - p)^T n_p$ 表示到切平面的距离。

隐式曲面

■ 从点云到隐式表面的转换:

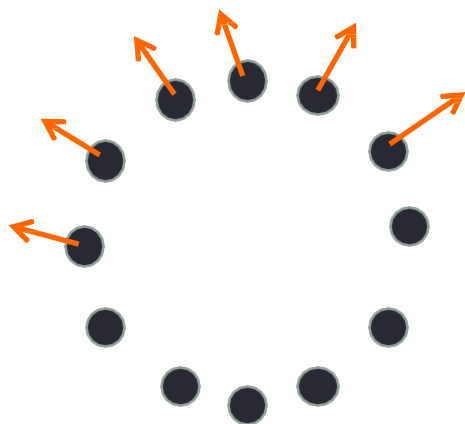


■ 最简单的方法:

1. 给定空间中的点 x , 找出点云数据中最近的点 p 。
2. 集合 $f(x) = (x - p)^T n_p$ 表示到切平面的距离。

隐式曲面

■ 从点云到隐式表面的转换:

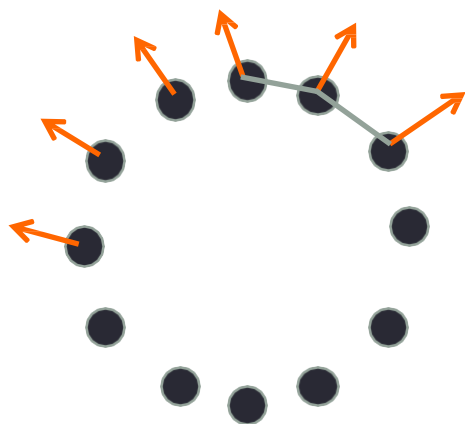


■ 最简单的方法:

1. 给定空间中的点 x , 找出点云数据中最近的点 p 。
2. 集合 $f(x) = (x - p)^T n_p$ 表示到切平面的距离。
3. 注意:需要一致定向的法线。PCA只给出方向上的法线

隐式曲面

■ 从点云到隐式表面的转换:

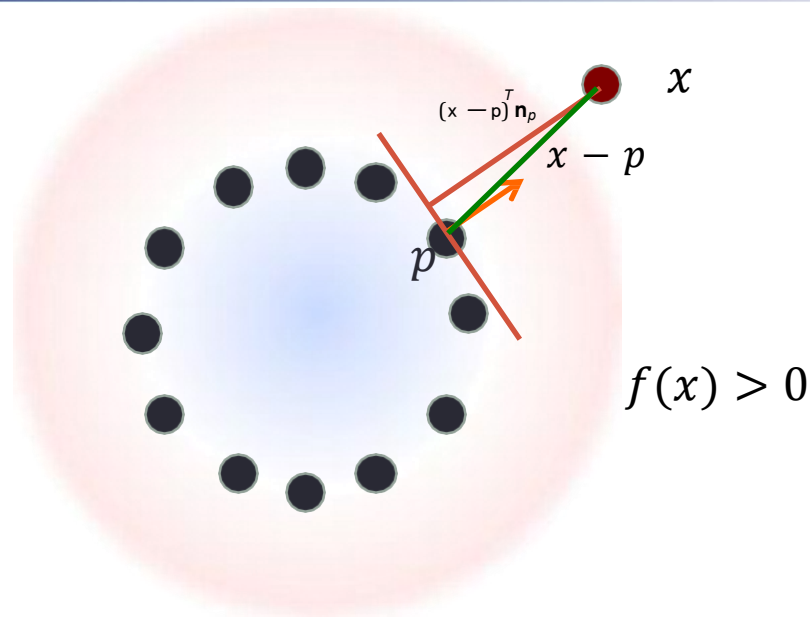


■ 最简单的方法:

1. 给定空间中的点 x , 找出点云数据中最近的点 p 。
2. 集合 $f(x) = (x - p)^T n_p$ 表示到切平面的距离。
3. 注意:需要一致定向的法线。一般来说, 比较难, 但可以尝试局部连接点和固定方向

隐式曲面

■ 从点云到隐式表面的转换:



■ 最简单的方法:

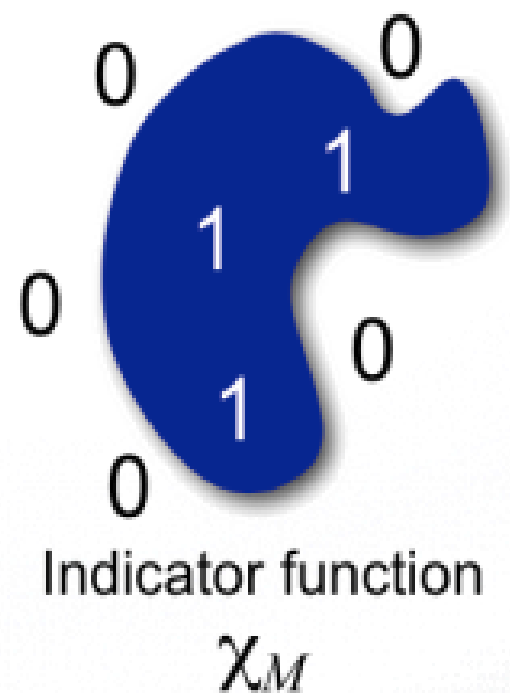
1. 给定空间中的点 x , 找出点云数据中最近的点 p 。
2. 集合 $f(x) = (x - p)^T n_p$ 表示到切平面的距离。

注意:距离函数在局部创建

泊松表面重建

- 核心思想:点云代表了物体表面的位置, 其法向量代表了内外的方向。通过隐式地拟合一个由物体派生的指示函数, 得到一个平滑表面的估计
- 主要目标:构建曲面的指标函数

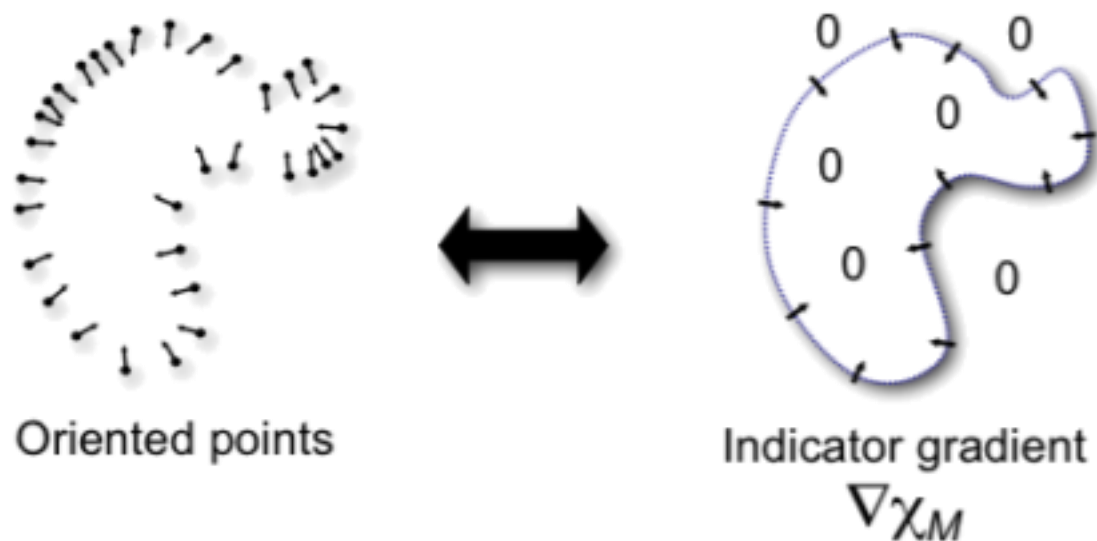
$$\chi_M(p) = \begin{cases} 1 & \text{if } p \in M \\ 0 & \text{if } p \notin M \end{cases}$$



Poisson surface reconstruction, Kazhdan, Bolitho, Hoppe, SGP'06

泊松表面重建

观察：



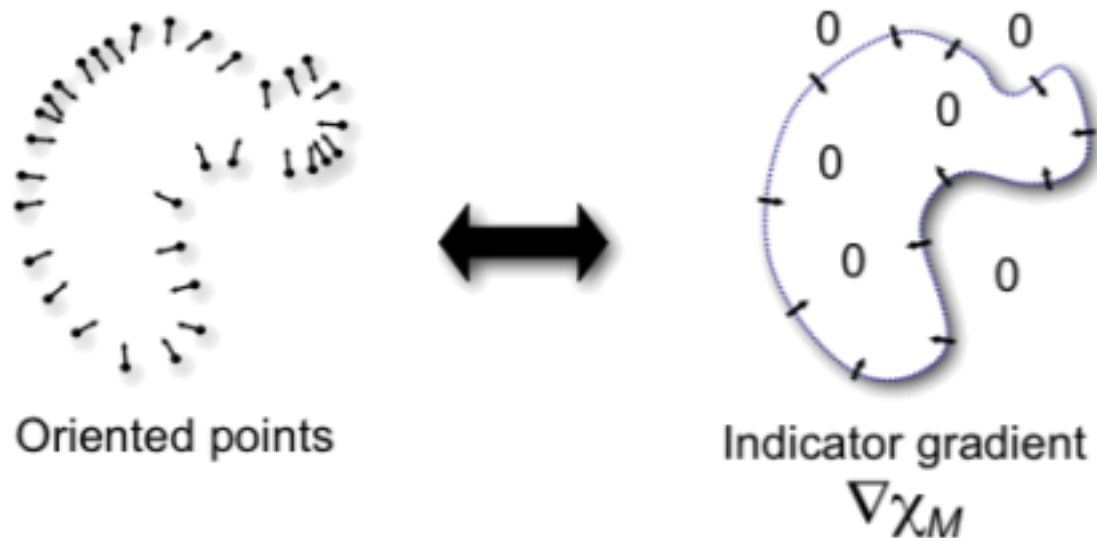
指标函数的梯度应与表面的法相向量场 V 一致。得到期望

$$E(\chi) = \int \|V - \nabla \chi_M(p)\|^2 dp$$

$\nabla \chi_M(p)$ 是 χ_M 在 p 处的梯度， V 是边界表面的法向

泊松表面重建

指示函数的求解

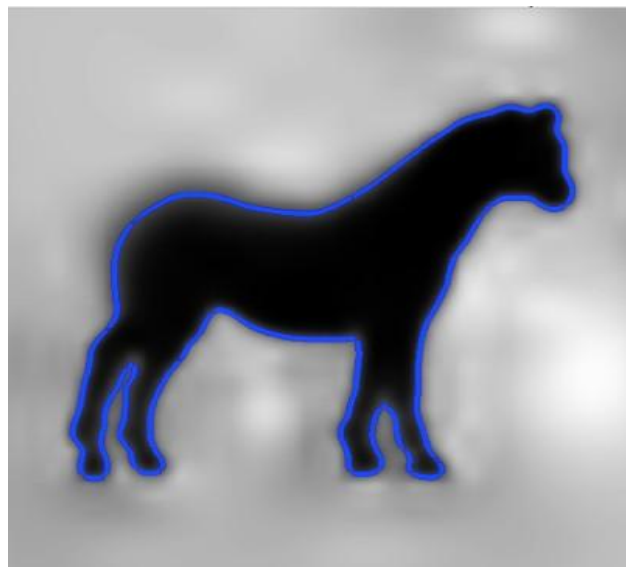
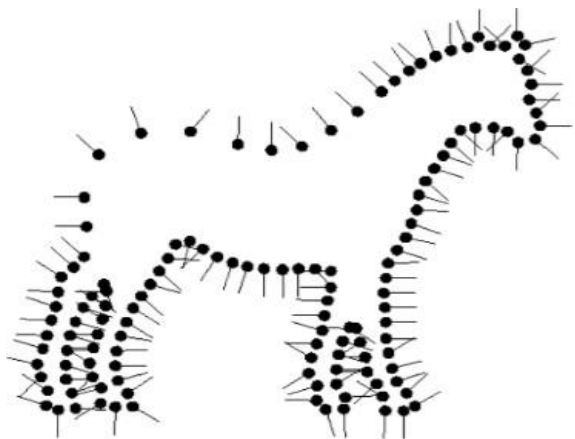


由于梯度场 V 通常不可积，我们无法通过方程 $\nabla \chi = V$ 来获取指示函数，因此在上述公式两边加一个散度，得到如下泊松方程：

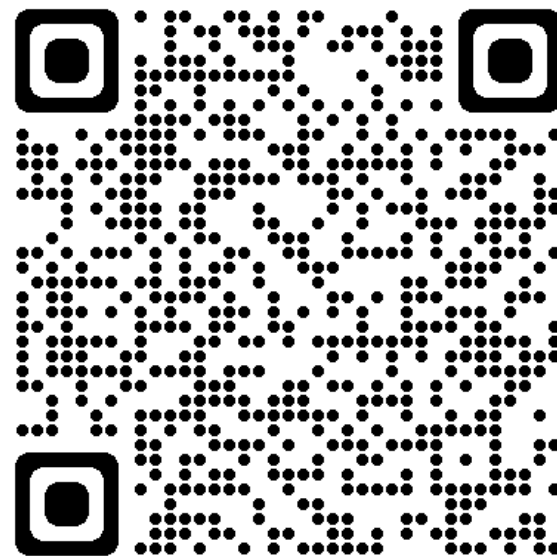
$$\Delta \chi = \nabla \cdot \nabla \chi, \quad \Delta \chi = \nabla \cdot V, \quad \min_{\chi} \|\Delta \chi - \nabla \cdot V\|^2$$

泊松表面重建

- 给定指标函数-提取等值面。

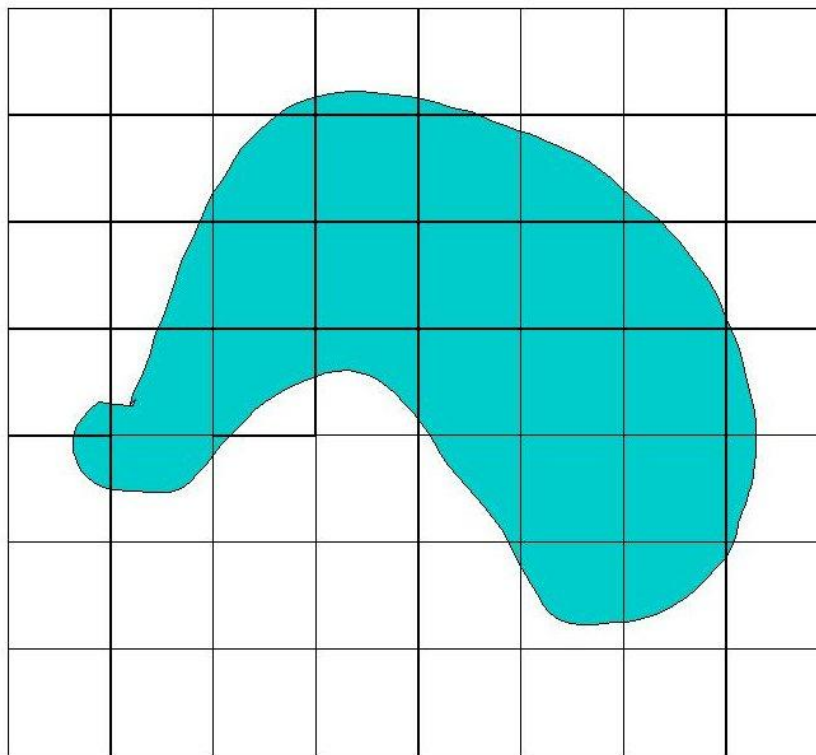
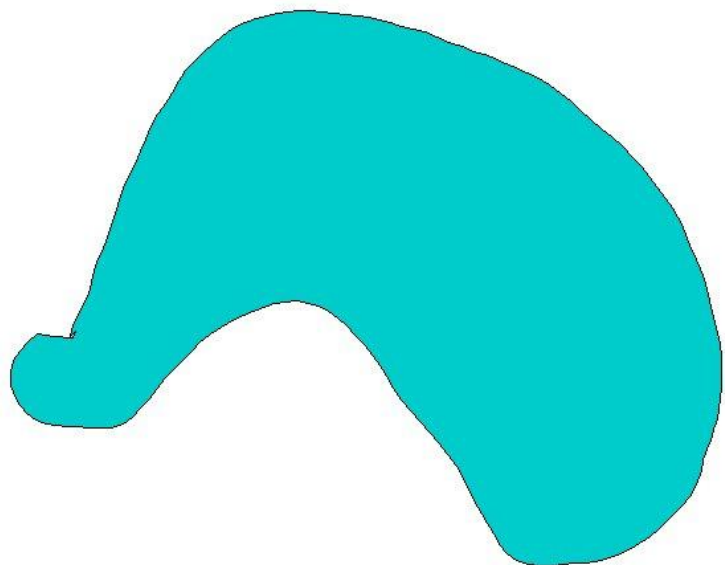


- 代码示例



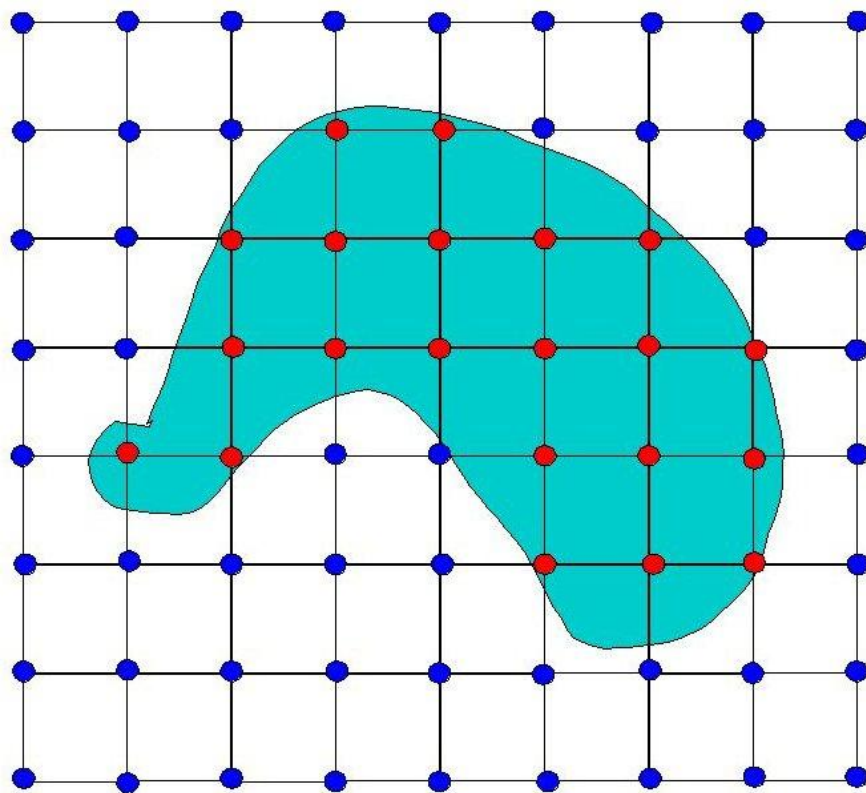
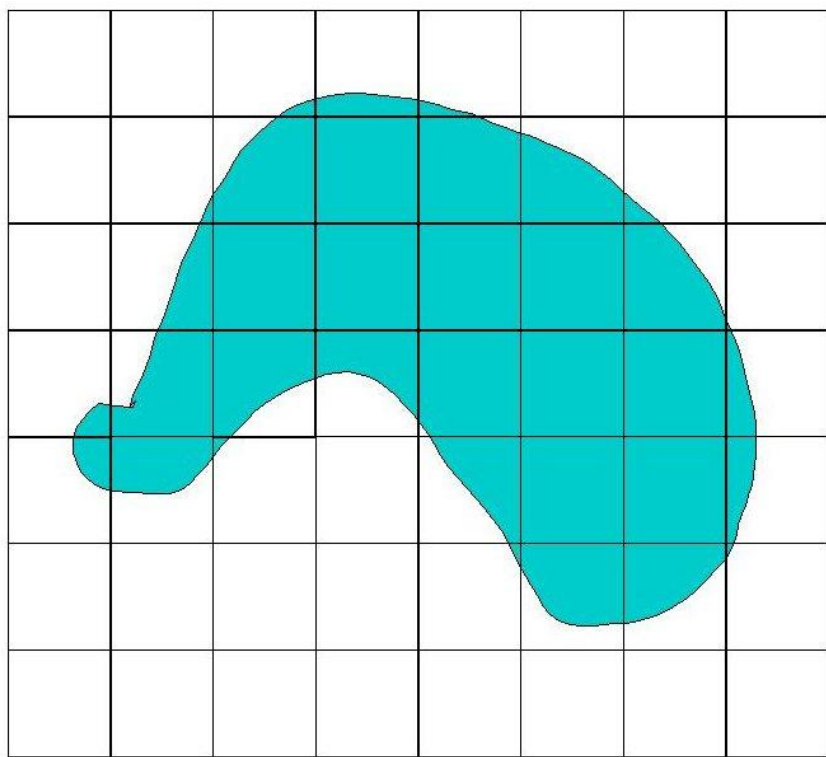
Marching Cube 2D情况

- 给定一个2D形状，我们可以把2D空间划分为一个个的像素。



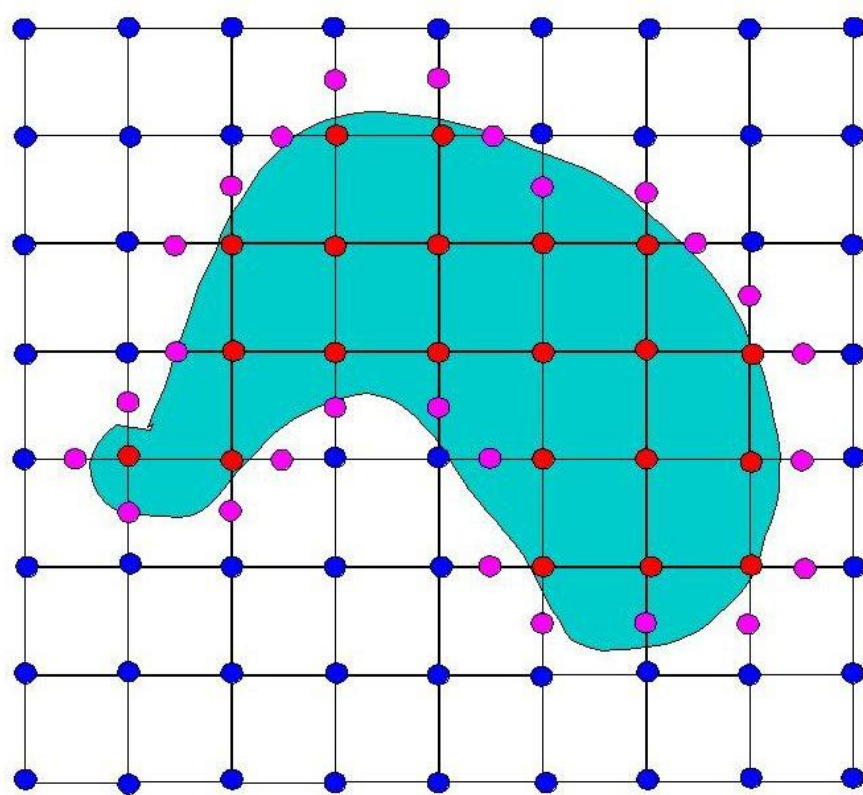
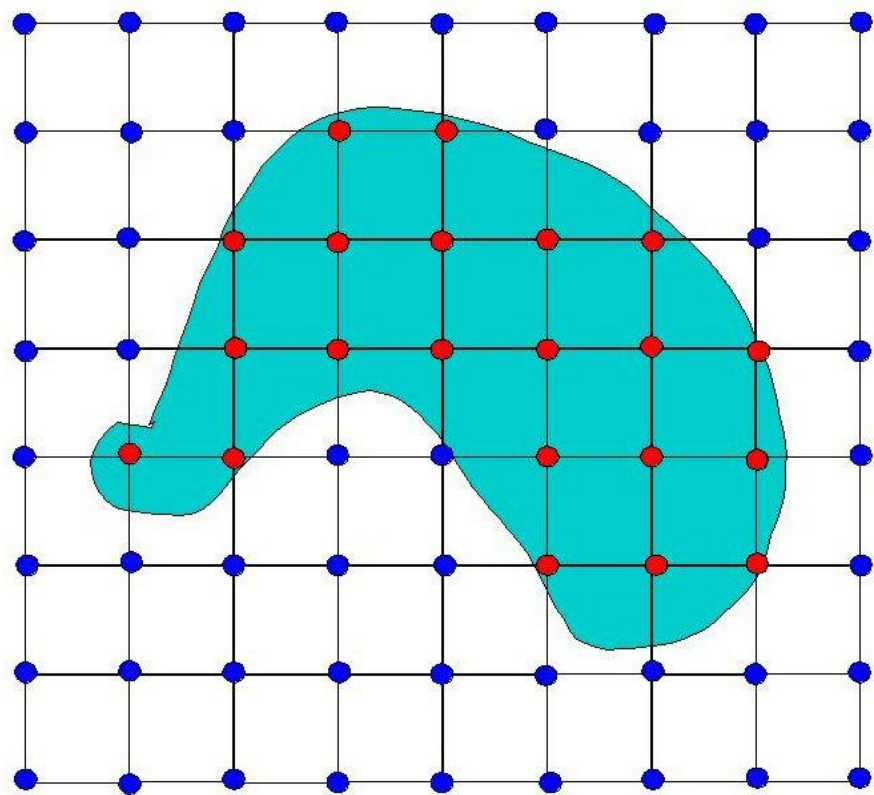
Marching Cube 2D情况

- 我们可以知道哪些格点在图形内部，哪些格点在图形外部。



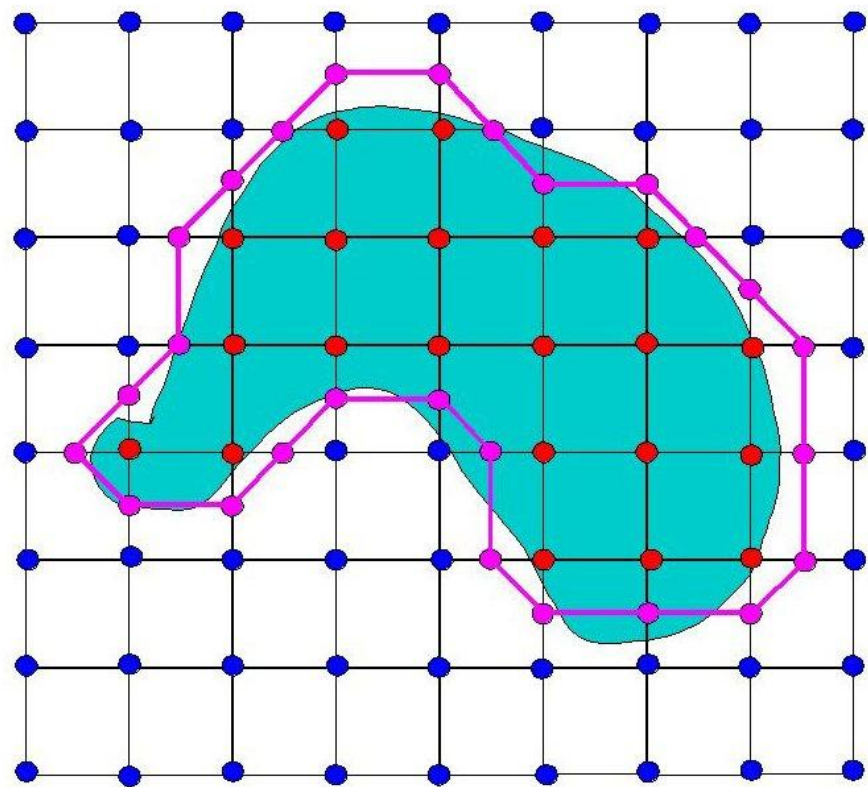
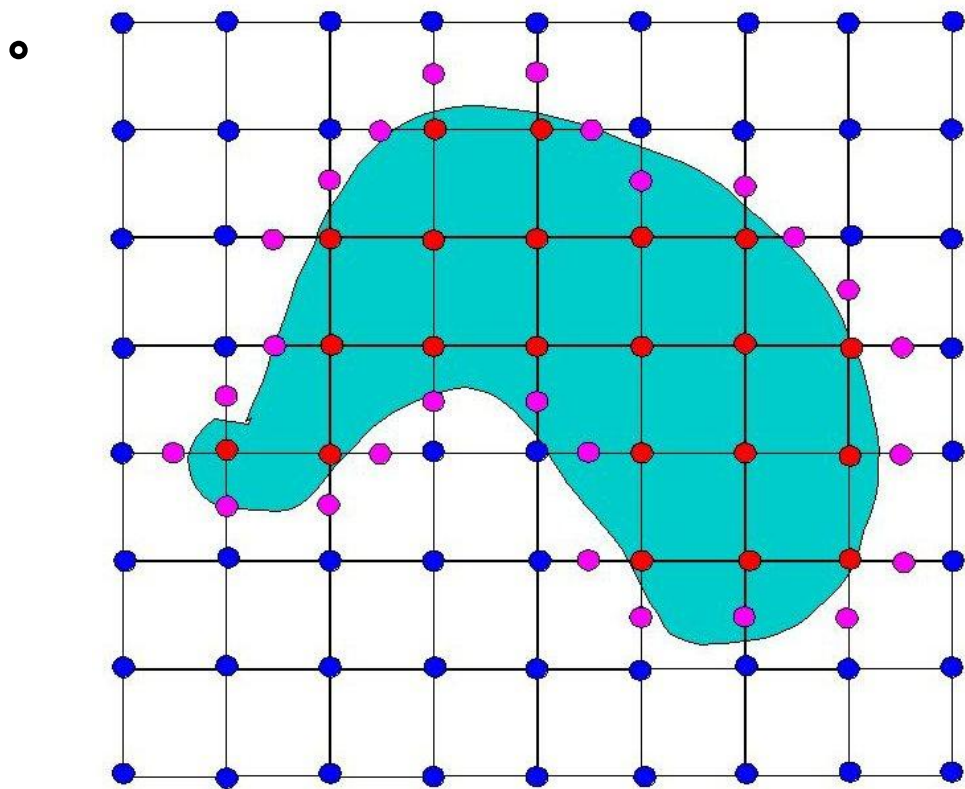
Marching Cube 2D情况

- 所以在内部和外部之间的每个边缘的某个地方，原始表面必须与网格相交（紫色点）



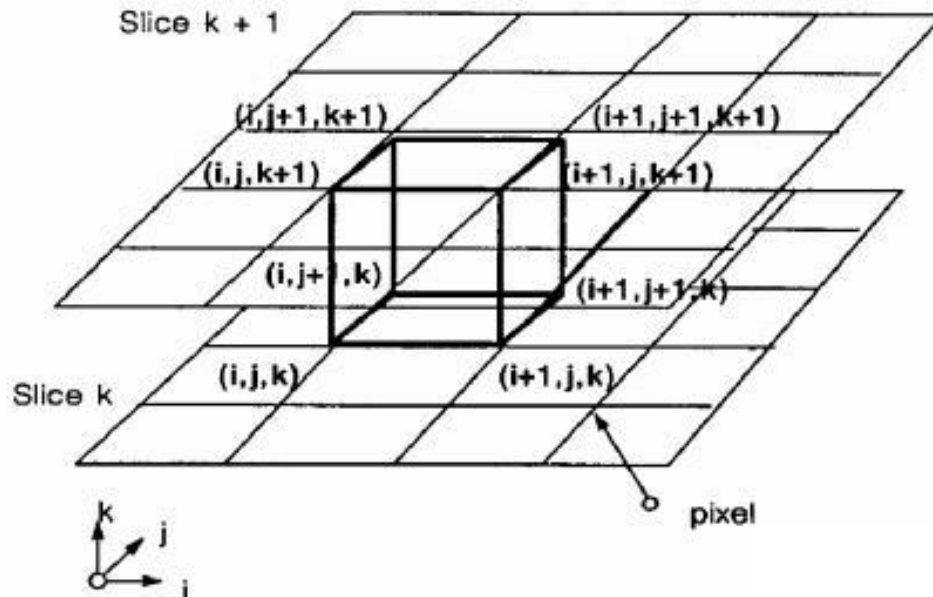
Marching Cube 2D情况

- 在每个正方形内，连接紫色点。将得到近似的原始表面（紫色线条）。这说明只需要给定表面与格点的交点即可得到原始表示的近似曲面。



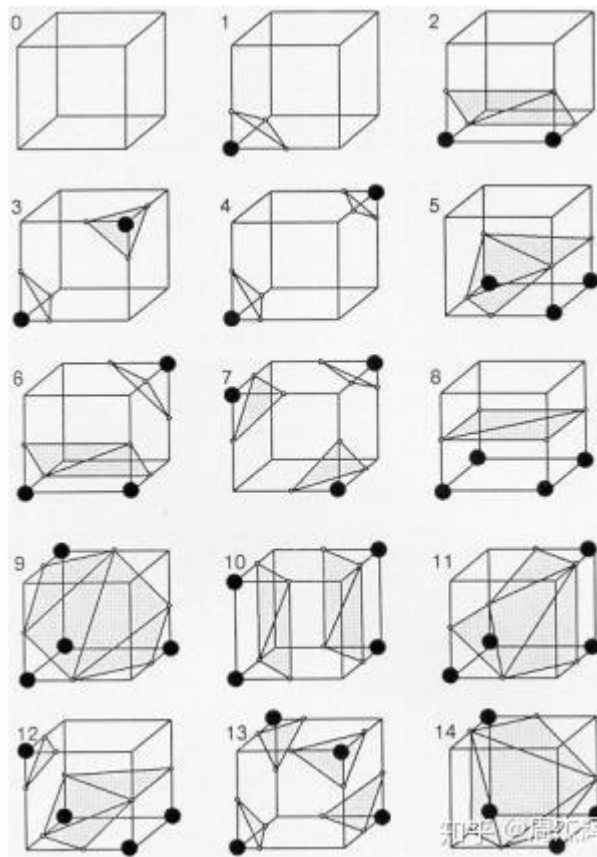
Marching Cube 3D情况

- 将空间划分为体素格
- 计算格点上的符号距离值
- 对于每个体素格，8个格点上的距离值正负情况共有 $2^8 = 256$ 种



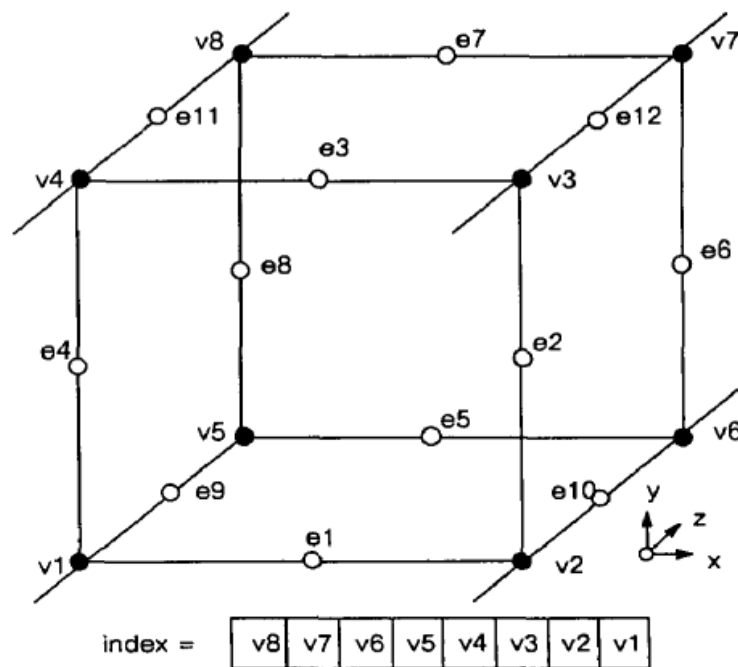
Marching Cube

- 但由于正负号反转状态不变，所以可以减少一半，为128种。
- 再根据旋转不变形，又可以减少到15种情况。



Marching Cube

- 根据每个顶点的状态，我们可以为每类制作一个8位索引，索引值共有256种，每一个对应一种上述的15中网格连接关系中的一种。
- 所以可以制作一个索引表表示从顶点状态到网格连接关系的映射。

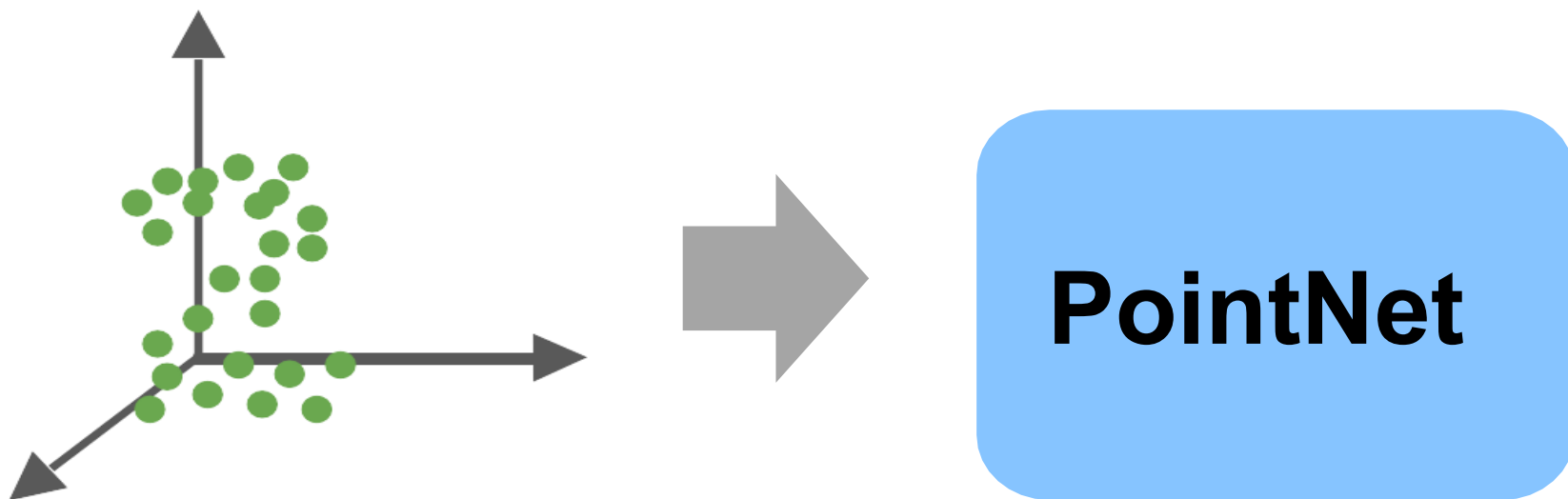


点云的特点

- 无序性，点云是一个点集，没有固定的顺序；
- 与邻域点又相互作用，点云中的个体不是独立的，与周围的点具相关，具有局域特征；
- 刚体旋转、平移不变性，旋转和平移不会改变点云的分类、分割结果

PointNet

- 可以直接输入无序点云进行处理



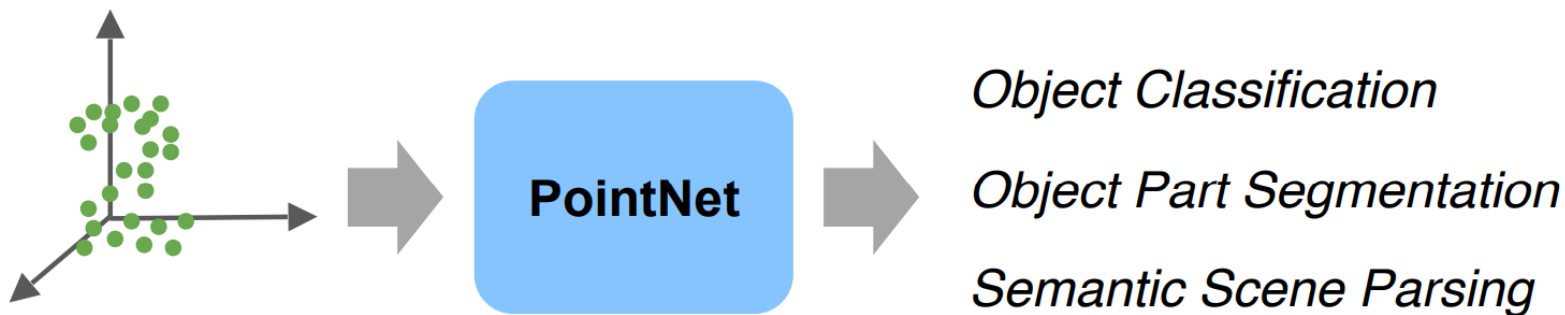
PointNet

- 可以直接输入无序点云进行处理
- 能够用于点云数据的多种认知任务，如分类、语义分割和目标识别



点云学习：PointNet

- PointNet是最早的几何学习方法之一，也是最经典的结构之一。
- 是一种端到端（end-to-end）的网络，适用于点云分类、分割多种任务的学习框架。

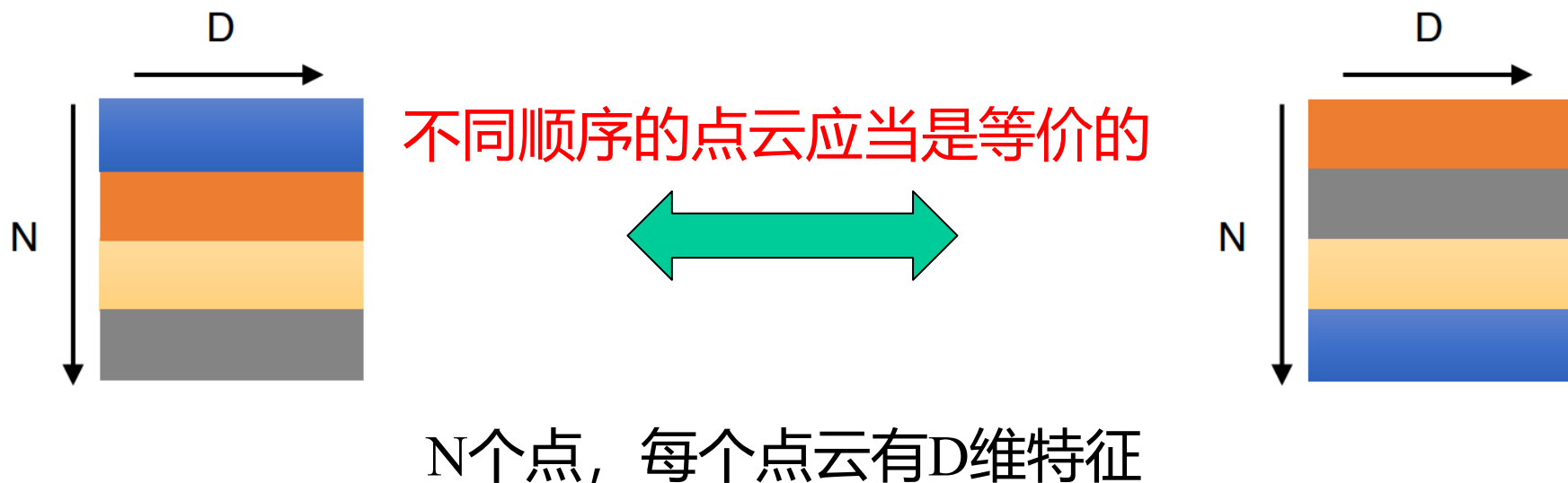


Qi, Charles R., et al. "Pointnet: Deep learning on point sets for 3d classification and segmentation." *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2017.

点云学习：PointNet

■ 点云的无序性：

- ◆ 点云可能有 $N!$ 种排列，一般网络难以学习
- ◆ 网络应当具有置换不变性



点云学习： PointNet

■ 对称函数:

◆ $(\pi_1, \dots, \pi_n)$ 是 $1 \dots N$ 的一个排列

$$f(x_1, x_2, \dots, x_n) \equiv f(x_{\pi_1}, x_{\pi_2}, \dots, x_{\pi_n}), \quad x_i \in \mathbb{R}^D$$

例子:

$$f(x_1, x_2, \dots, x_n) = \max\{x_1, x_2, \dots, x_n\}$$

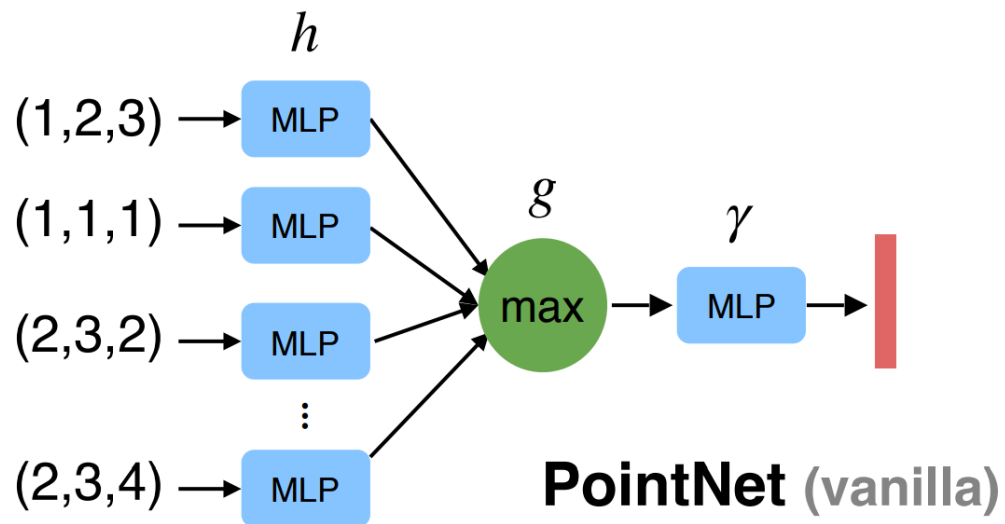
$$f(x_1, x_2, \dots, x_n) = x_1 + x_2 + \dots + x_n$$

...

点云学习：PointNet

■ PointNet 网络架构(简单版)

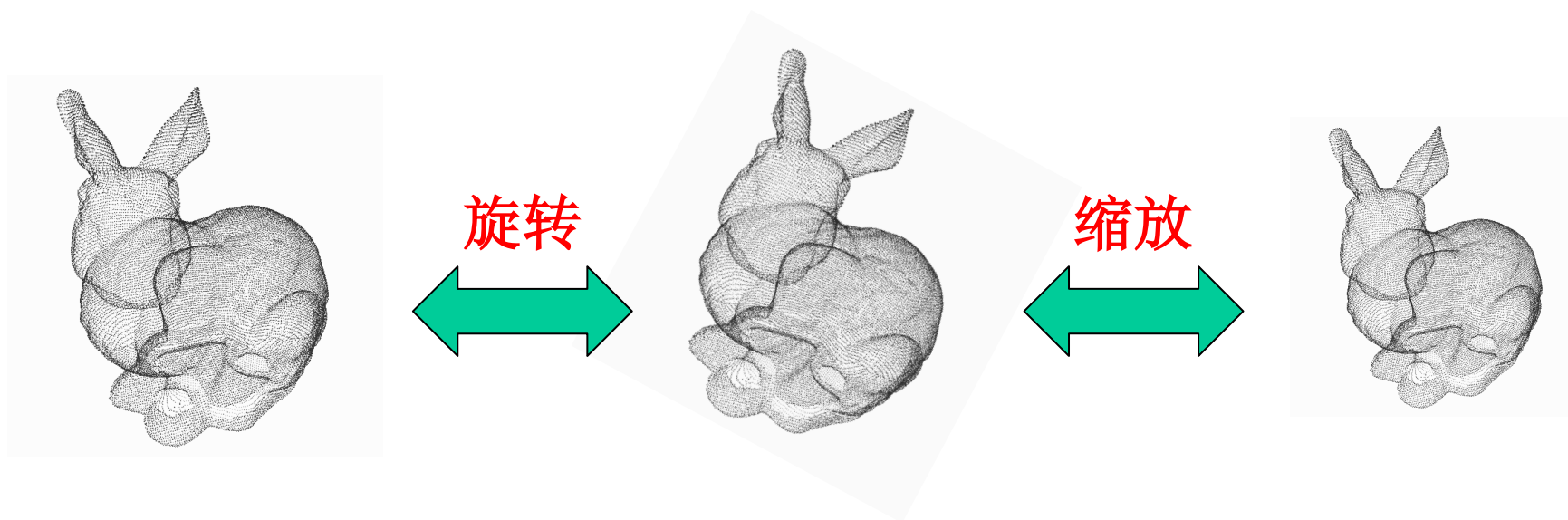
- ◆ 对每个点，使用共享权重的MLP（多层感知机）提取特征
- ◆ 逐维度取Max（对称函数）进行特征聚合



点云学习：PointNet

■ 几何变换不敏感：

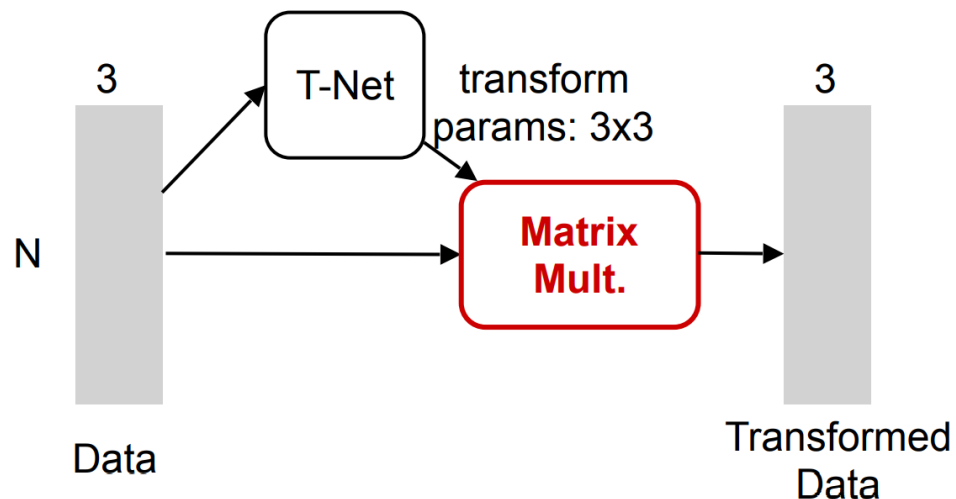
- ◆ 模型对点云平移、旋转、缩放后的输出结果应当相似。



点云学习：PointNet

■ 对齐模块 T-Net:

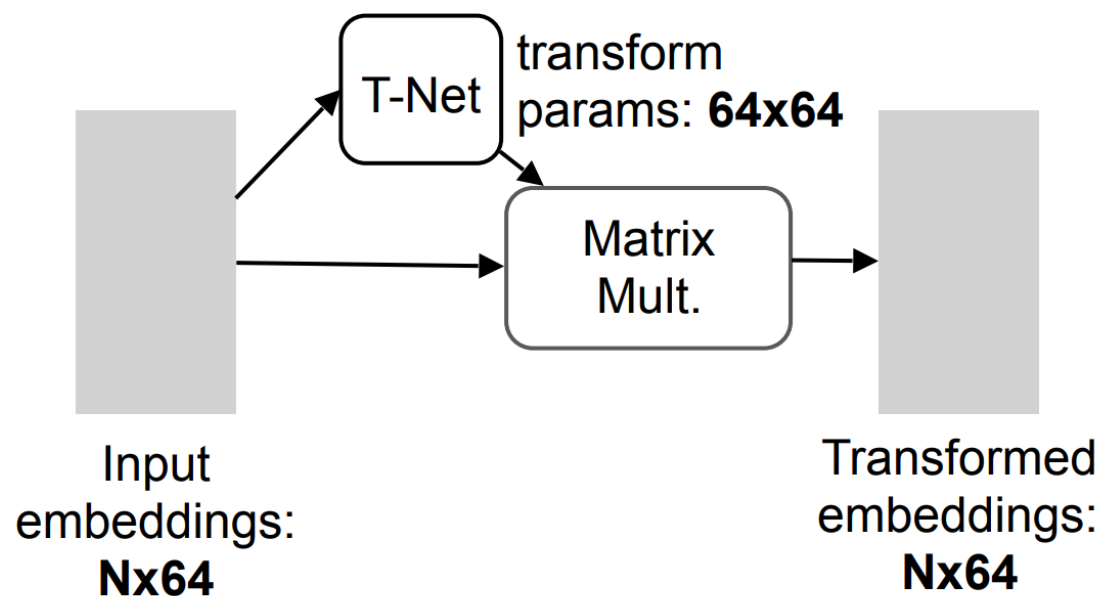
- ◆ 加入到数据与MLP之间，对输入数据几何变换
- ◆ 对齐输入，使得网络满足几何变换不变性
- ◆ 变换参数由网络训练得到



点云学习：PointNet

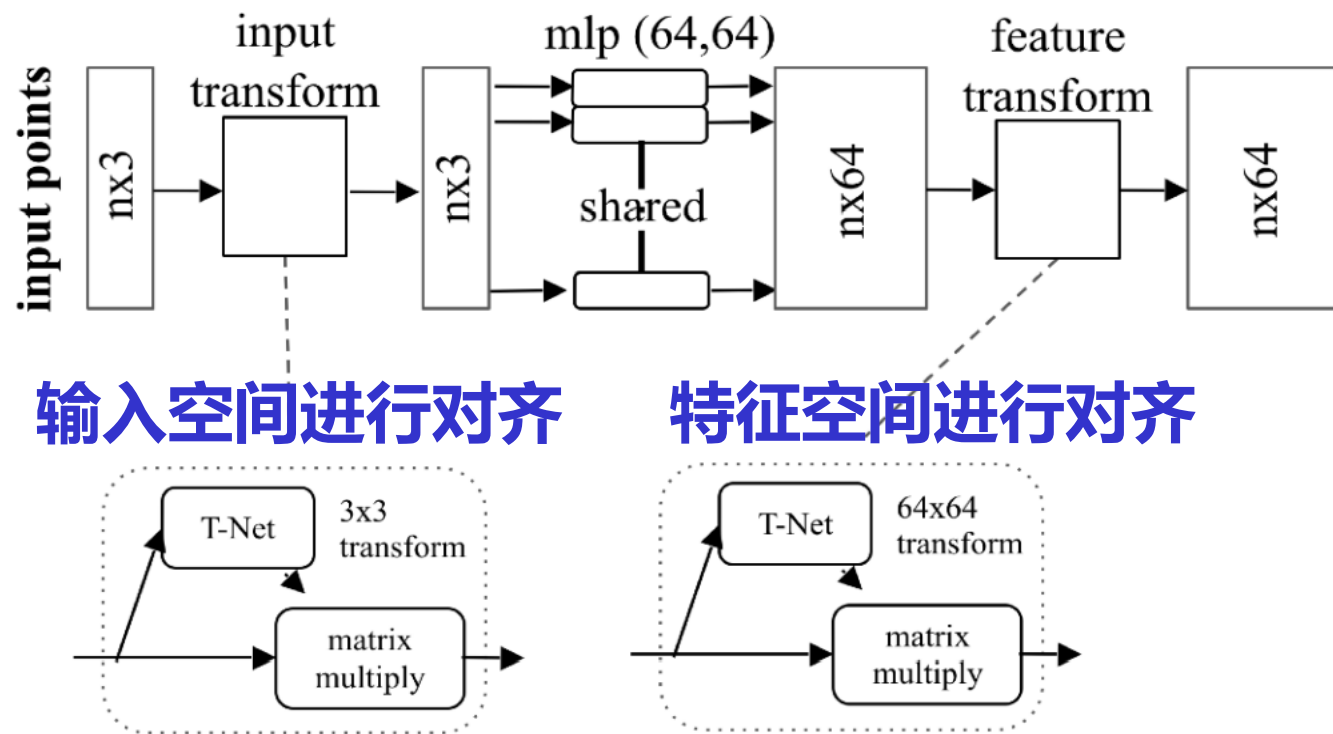
■ 对齐模块 T-Net:

- ◆ 加入到 MLP 之间，对齐网络特征空间
- ◆ 变换参数由网络训练得到



PointNet-分类

- PointNet 分类网络：前半段
 - ◆ 由多组 T-Net 和 MLP 相间构成

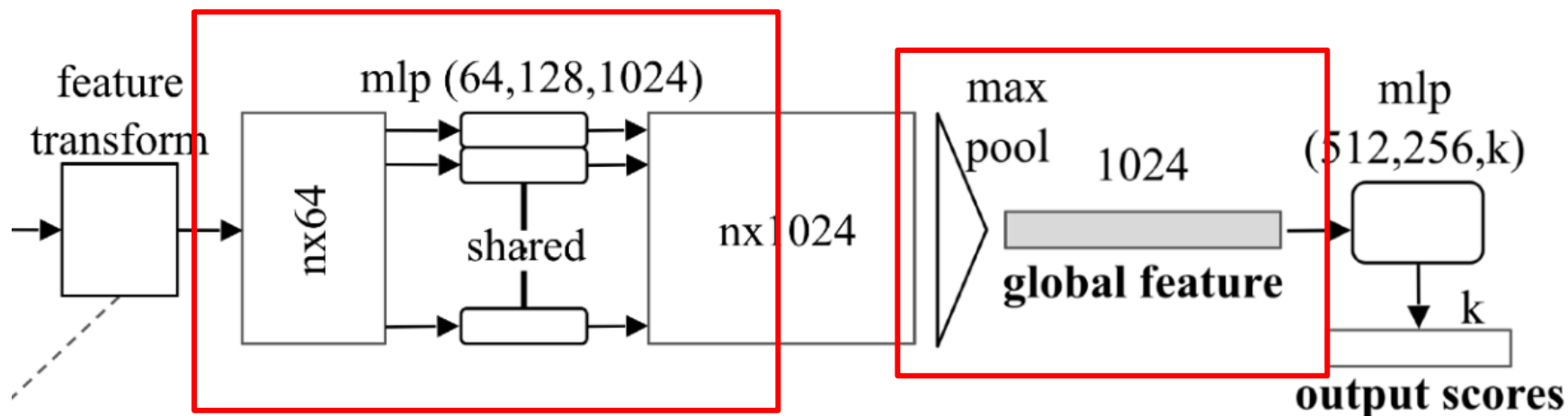


PointNet-分类

■ PointNet 分类网络：后半段

- ◆ MaxPool聚合得到1024维特征向量
- ◆ 经过全连接网络得到分类结果

利用对称函数进行特征聚合



对齐后的特征再次进行学习

PointNet-分类

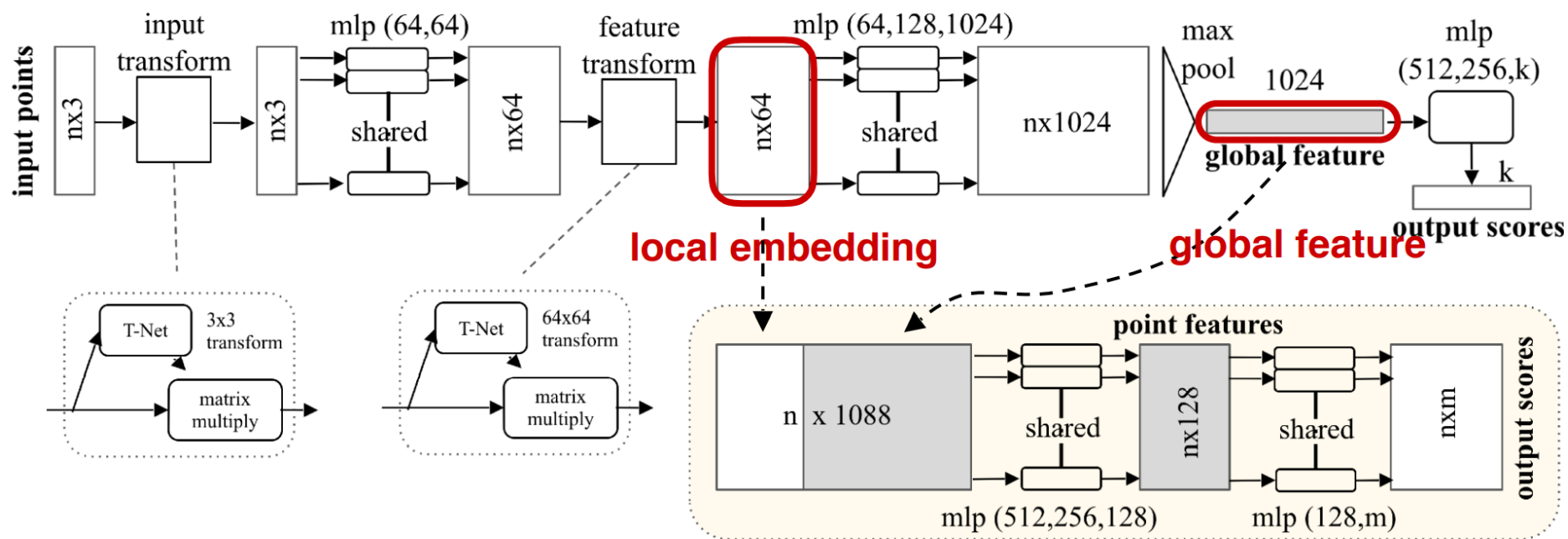
3D CNNs

	input	#views	accuracy avg. class	accuracy overall
SPH [12]	mesh	-	68.2	
3DShapeNets [29]	volume	1	77.3	84.7
VoxNet [18]	volume	12	83.0	85.9
Subvolume [19]	volume	20	86.0	89.2
LFD [29]	image	10	75.5	-
MVCNN [24]	image	80	90.1	-
Ours baseline	point	-	72.6	77.4
Ours PointNet	point	1	86.2	89.2

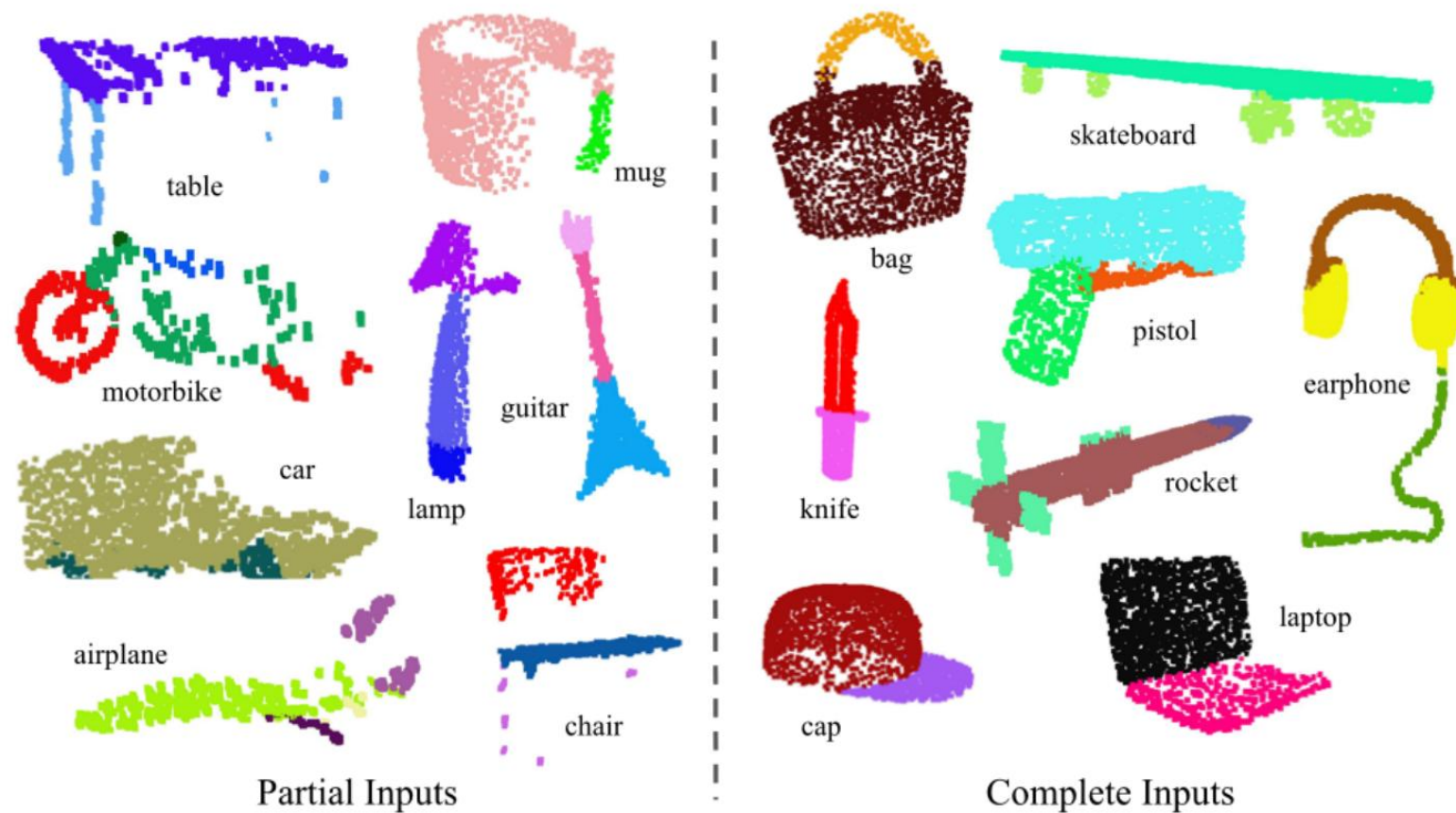
分类结果

PointNet-分割

- 全局特征(global feature)与局部特征(local embedding)进行组合
- 使用组合后的特征预测每个点的类别

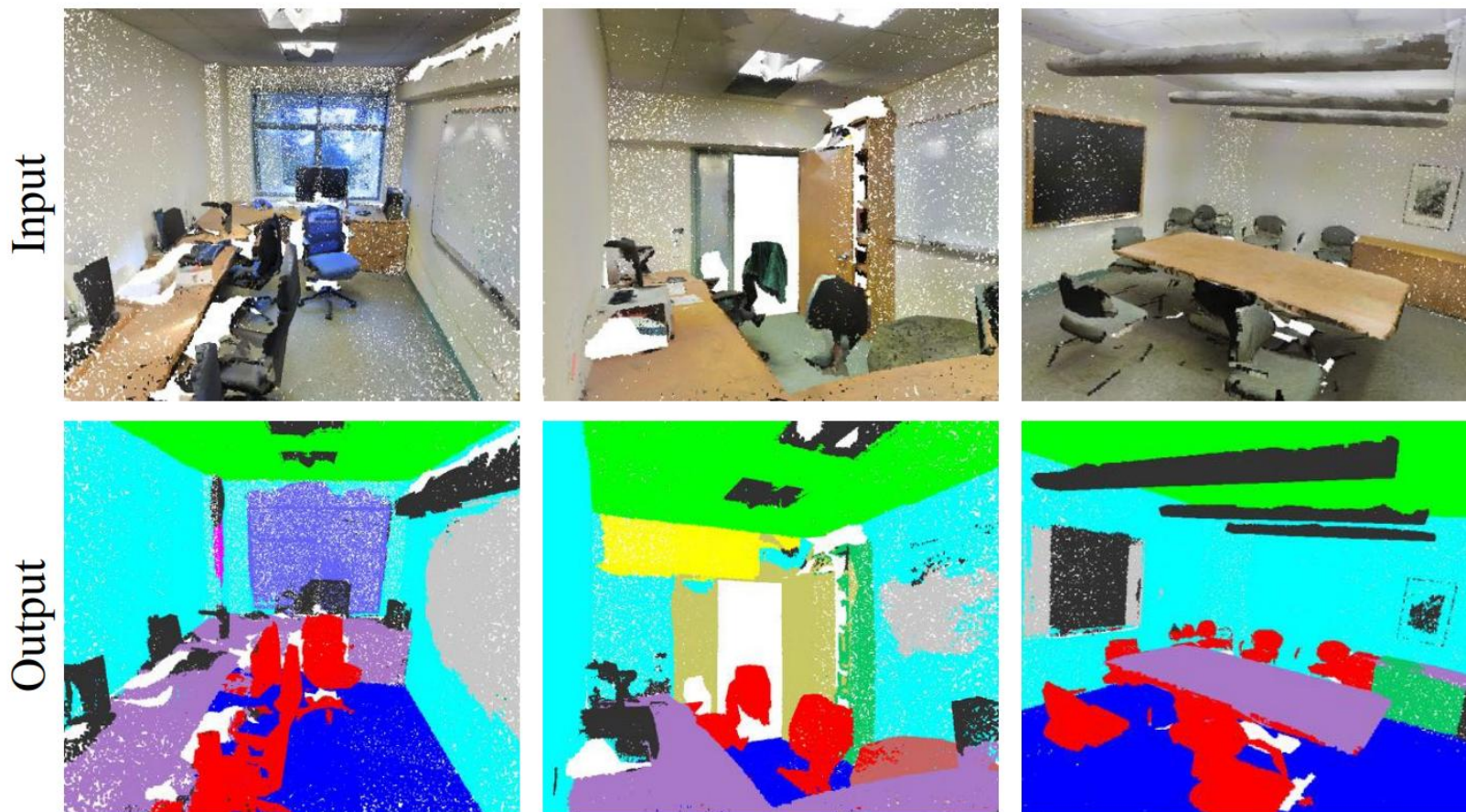


PointNet-分割



部件分割结果

PointNet-分割



场景语义分割结果

谢谢！ 欢迎交流！

高林

gaolin@ict.ac.cn